🛡️ Module 11: Reverse Engineering, Fuzzing and Binary Exploitation

```
# Exercise 03: Exploring Binary Analysis Methodology
```

Scenario

Binary Analysis Methodology encompasses a structured workflow for discovering, categorizing, inspecting, and reverse engineering executable binaries on a target Linux system. The analysis pipeline transitions through four key phases:
1. **Discovery & Classification**: Finding executables via `find / -executable -type f`, identifying MIME encoding with `file -i`, locating binaries using `updatedb` / `locate`, and tracking active processes via **ps -ef**.
2. **Static Property Inspection**: Extracting embedded ASCII strings using `strings`, inspecting section headers, program headers, relocation tables, and symbol tables using `readelf` (`-h`, `-l`, `-S`, `-s`, `-x .text`), dumping raw hex via `hexdump -C` or `xxd`, and disassembling machine code using `objdump -d -M intel` or `ndisasm`.
3. **Dynamic Debugging & Disassembly**: Loading binaries into debuggers (**GDB**, **Evan's Debugger** `edb`), configuring Intel disassembly flavor (**set disassembly-flavor intel**), setting breakpoints at `main`, single-stepping instructions via `nexti`, inspecting instruction pointer registers (`x/s $rip`), and mapping function control-flow graphs.
4. **System & Library Call Tracing**: Monitoring kernel-level system calls using **strace** and tracing dynamically linked C library function calls (such as `strcmp`, `printf`, `scanf`) using **ltrace**.

In this exercise, we log into **Software-Test-Linux** (**student** / **studentpassword**), elevate privileges via **sudo -i**, enumerate all system executables, identify mime types, view active process listing (**ps -ef**), perform deep static analysis on sample 64-bit binaries (**info** and **info2**), debug execution flow in GDB and `edb`, generate function graphs, and trace system calls (`strace`) and library calls (`ltrace`).

Objective

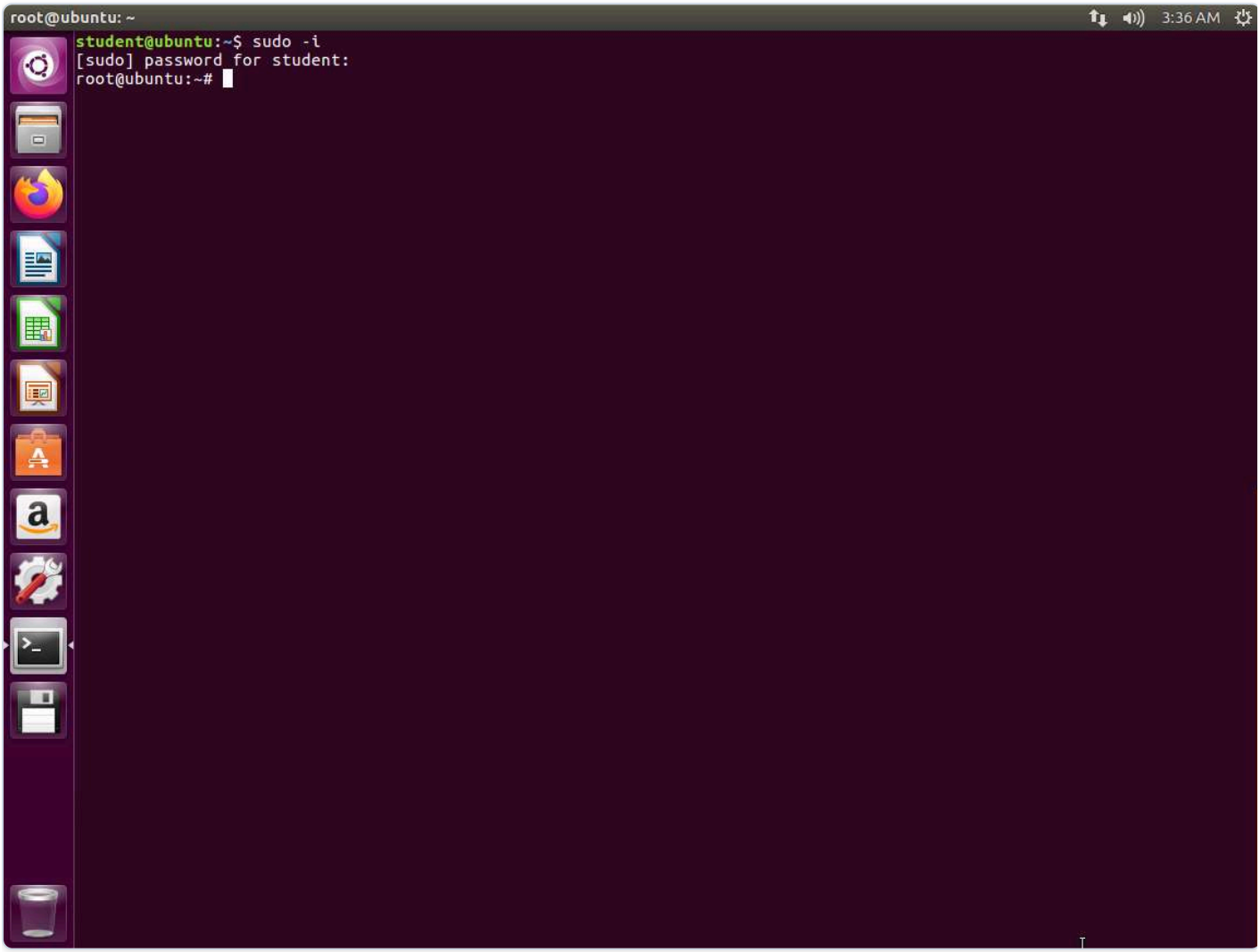
The objective of this lab is to help students learn how to:
- Authenticate to 64-bit Linux target environment (**student** / **studentpassword**) and elevate to root (**sudo -i**)
- Enumerate executable binaries across the filesystem using **find / -executable -type f**
- Inspect MIME content types using **file -i /bin/cat** and automated loop filtering
- Query all active system processes using full-format process listing command: **ps -ef**
- Extract embedded hardcoded secrets and string tables using **strings ~/examples/samplecode/info**
- Perform comprehensive ELF header, segment, section, and symbol table auditing via `readelf` (`-h`, `-l`, `-S`, `-p .text`, `-x .text`, `-R .text`, `-p .strtab`)
- Perform disassembly using `objdump -d -M intel`, `ndisasm`, and `hexdump -C`
- Dynamically debug binaries in GDB (**gdb ~/examples/samplecode/info**), configure Intel syntax (**set disassembly-flavor intel**), step instructions (**nexti**), and inspect RIP (**x/s $rip**)
- Graph function call trees in Evan's Debugger (**edb --run ~/examples/samplecode/info**)
- Intercept OS system calls and C library string comparisons using **strace** and **ltrace**

Task 1: Perform System Discovery, Static Binary Inspection, Dynamic Debugging, and Call Tracing

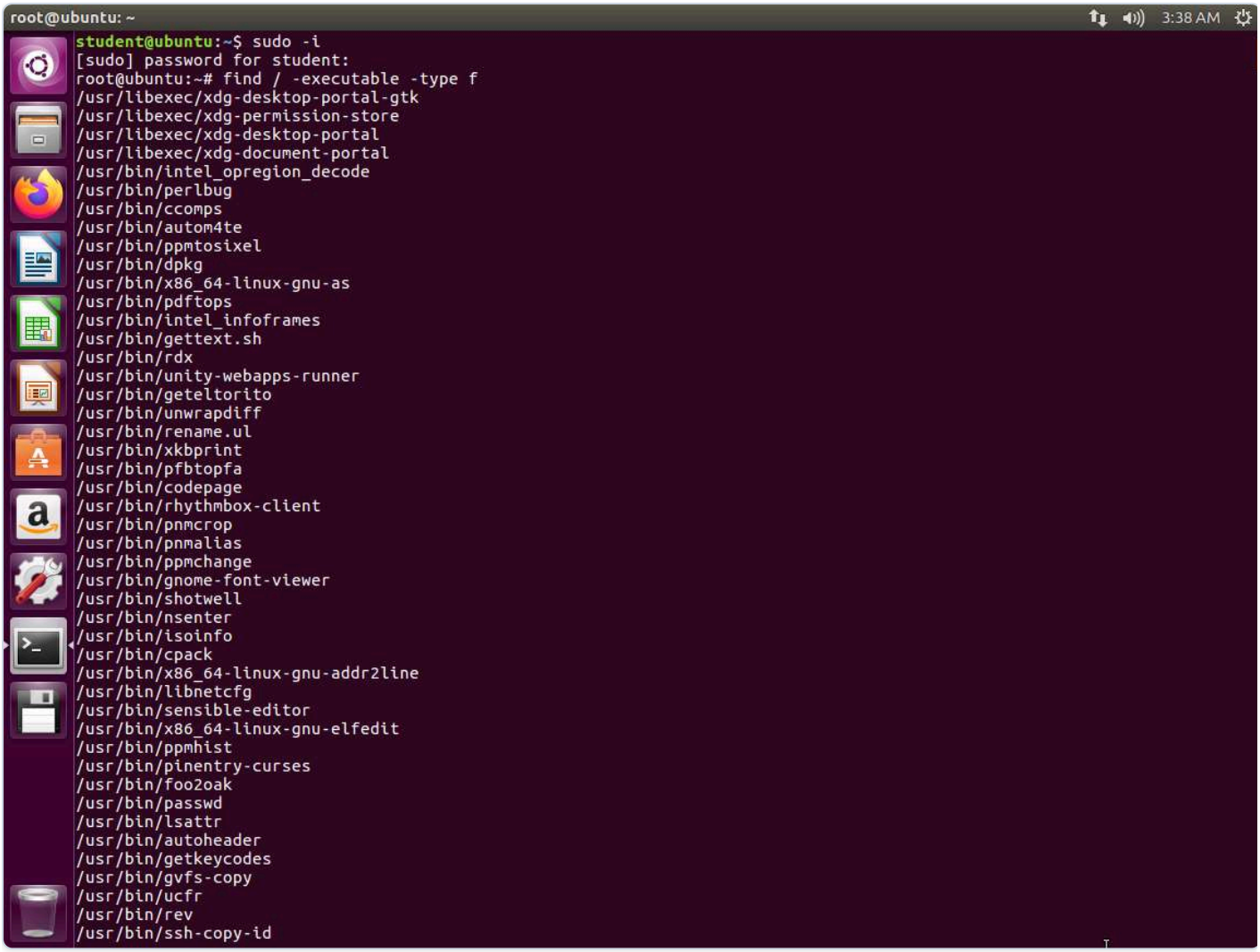
In this task, we elevate to root, discover executable binaries, view running processes (`ps -ef`), analyze sample binaries `info` and `info2` using `strings` , `readelf` , `objdump` , `gdb` , `edb` , `strace` , and `ltrace` .

Lab Tasks

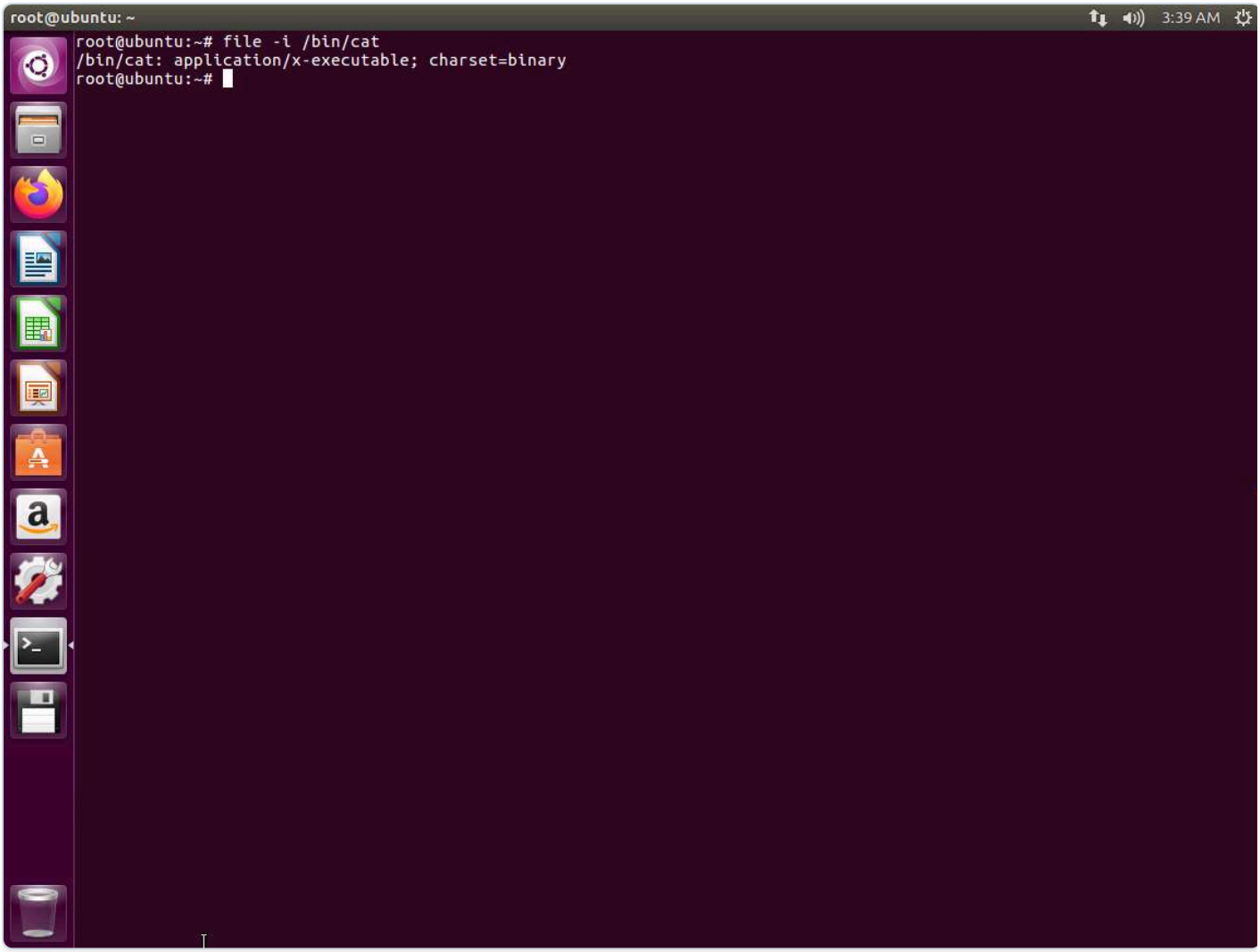
1. Log into **Software-Test-Linux** VM using credentials (**student** / **studentpassword**) and elevate to root:
   sudo -i



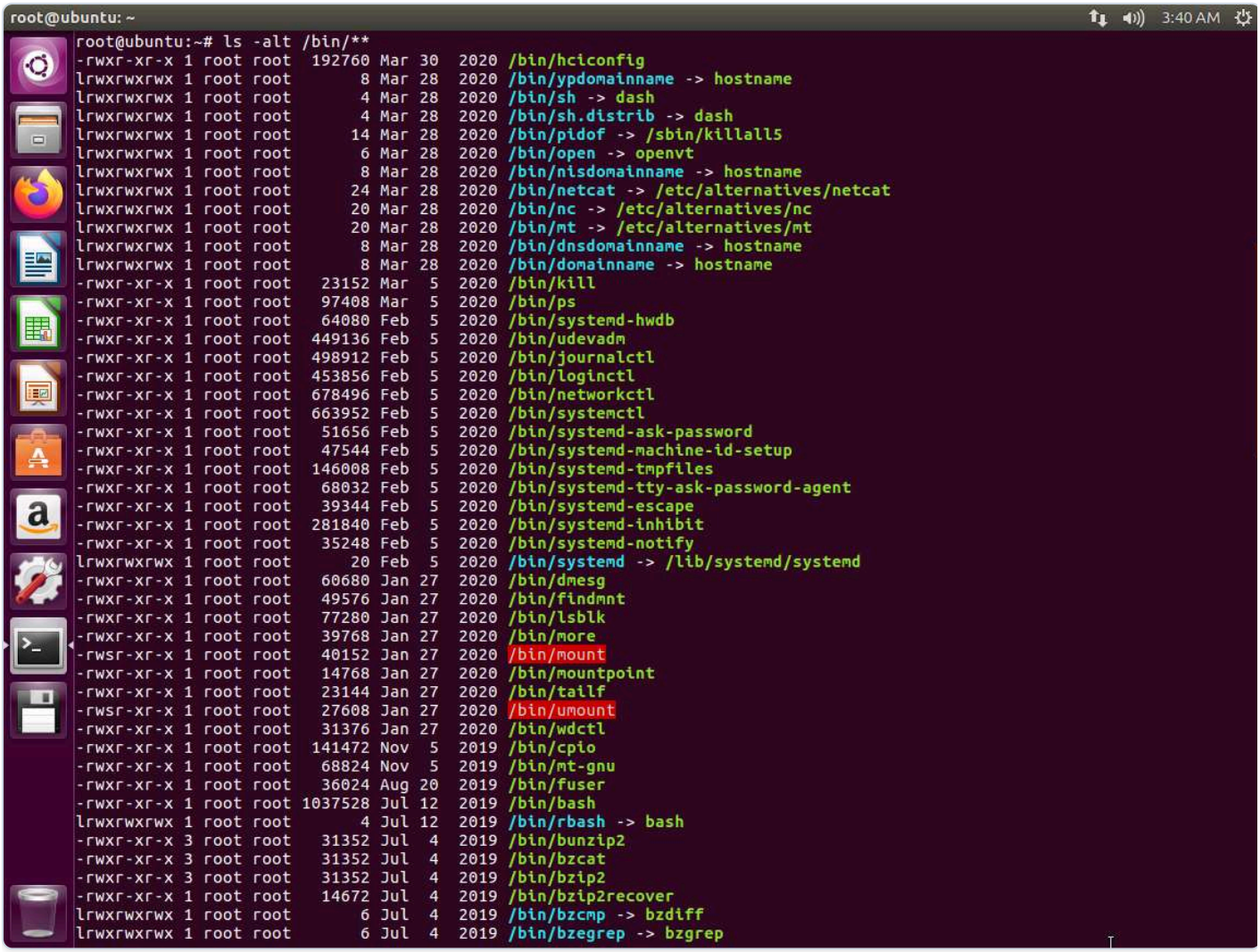
1. Discover executable files across the filesystem:
   find / -executable -type f



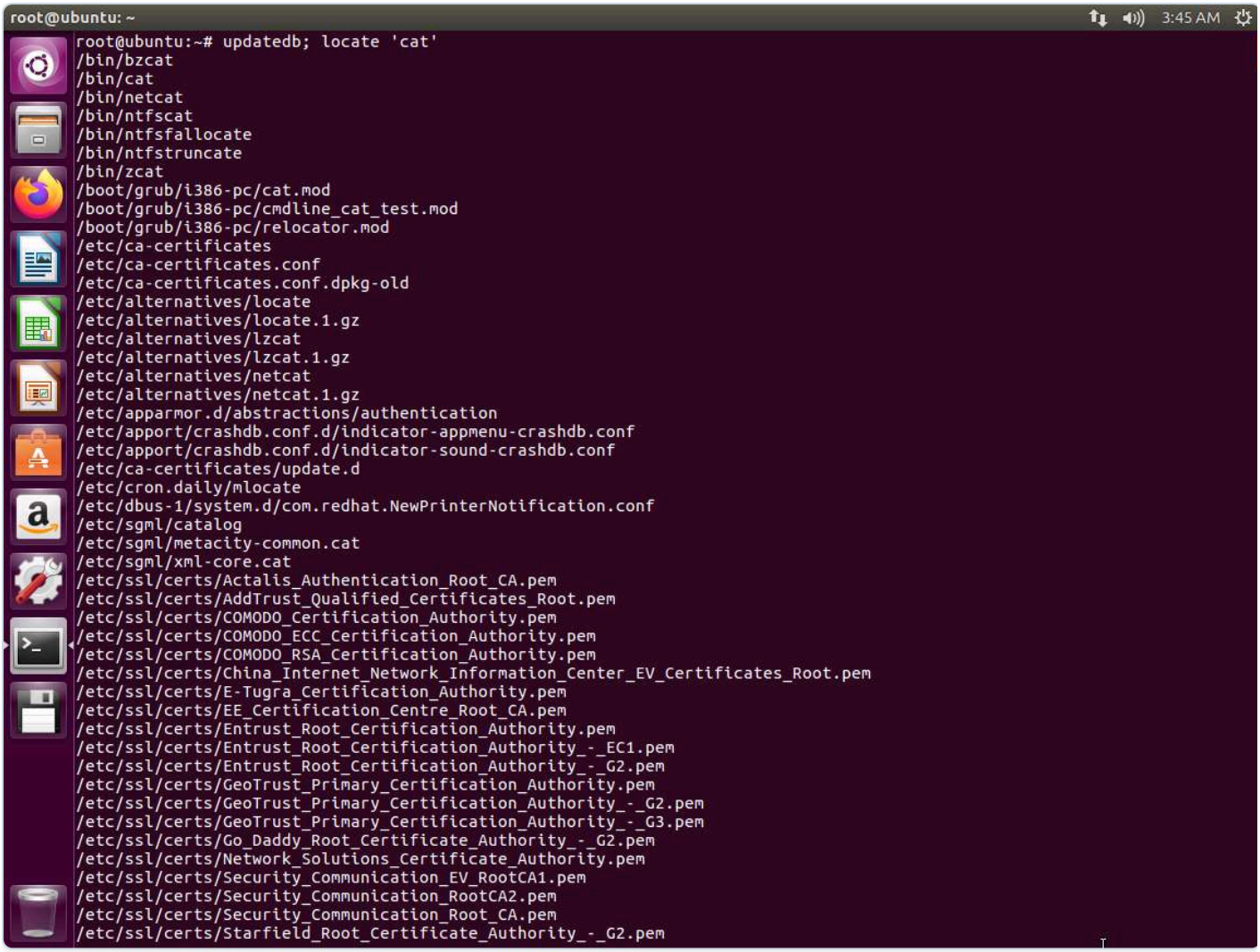
1. Inspect MIME type of `/bin/cat` binary:
   file -i /bin/cat



1. List binaries in `/bin/` directory sorted by modification time:
   ls -alt /bin/

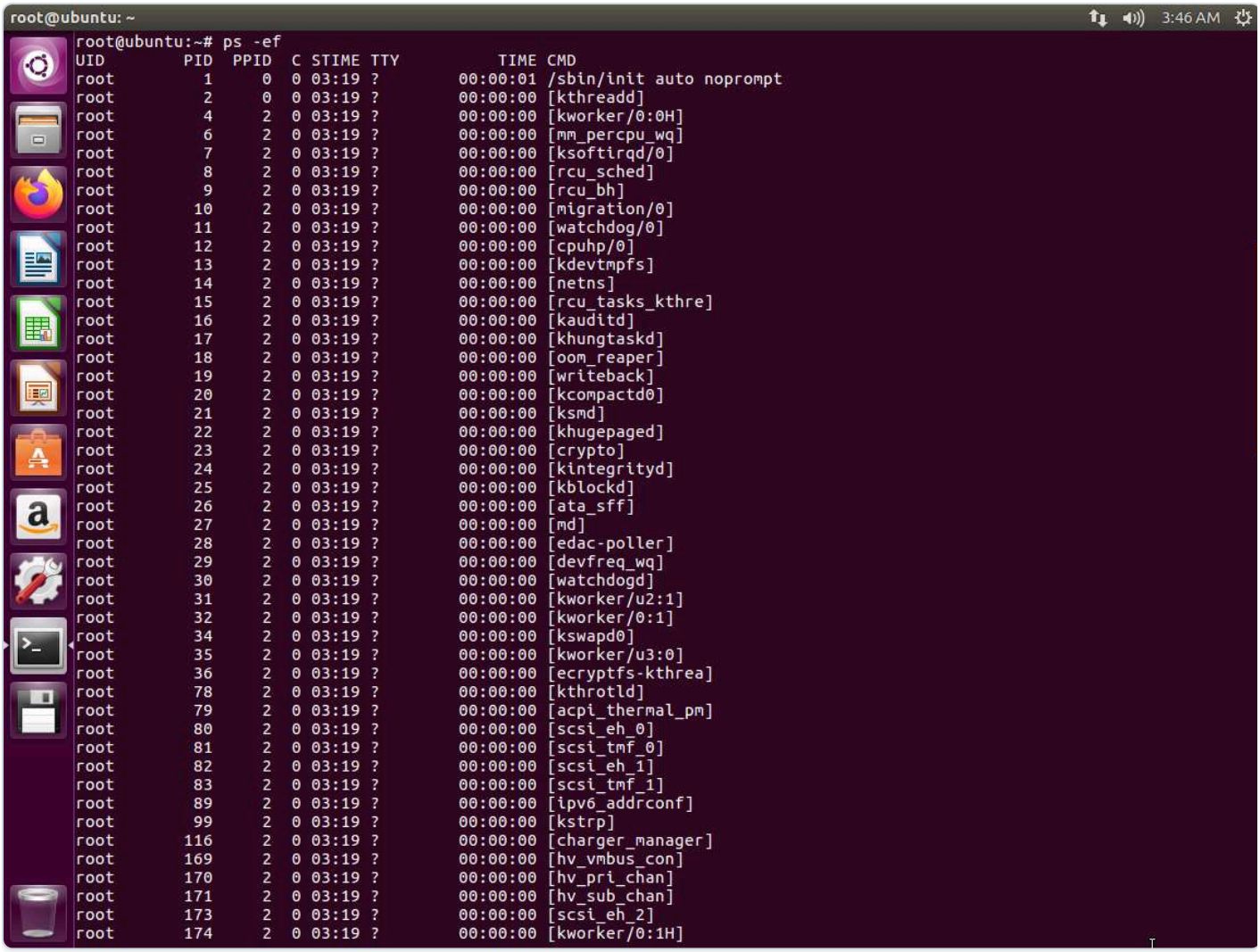


1. Update file location database and locate binaries matching `cat`:
   updatedb; locate 'cat'

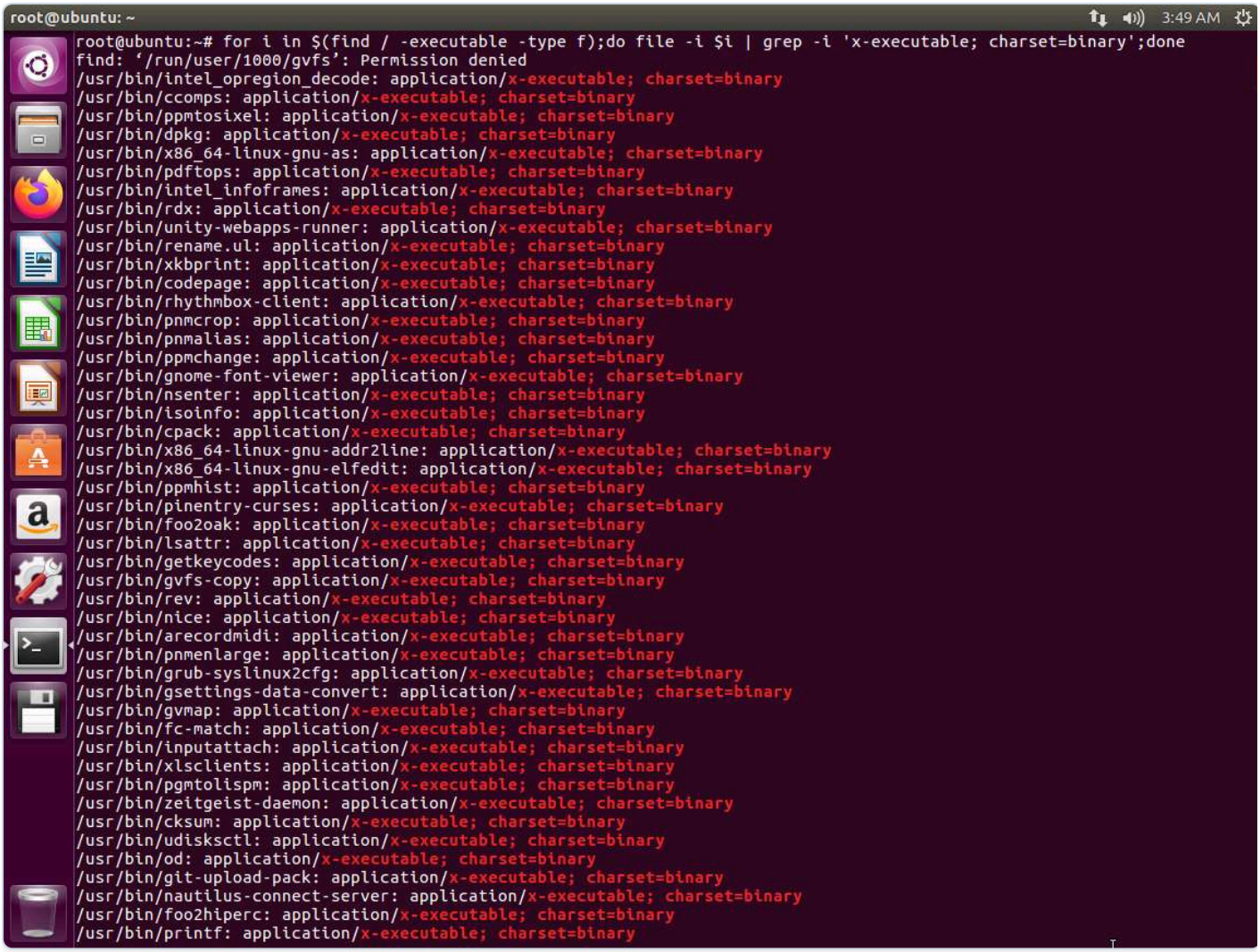


1. Display all running system processes across all users in full format:

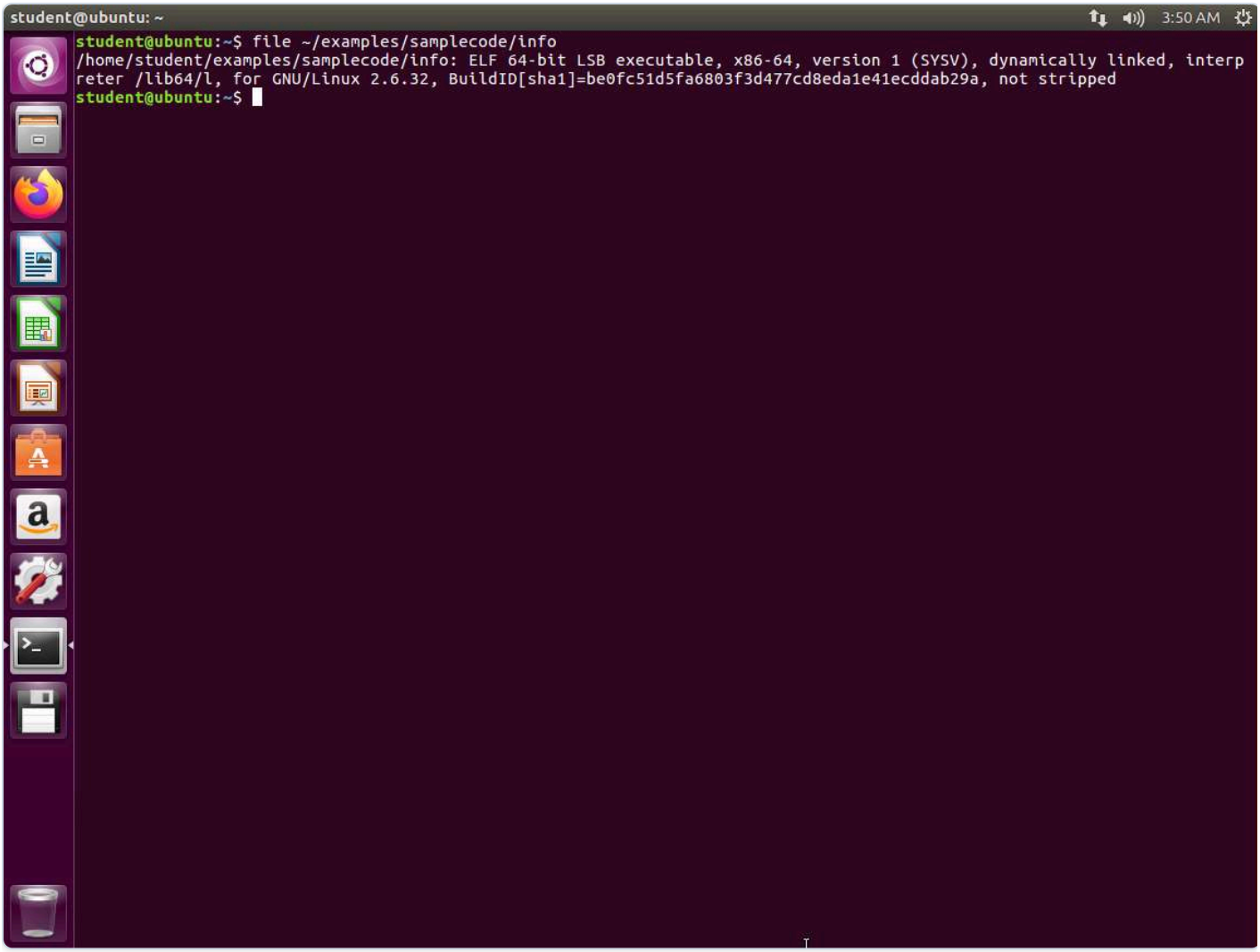
   ps -ef



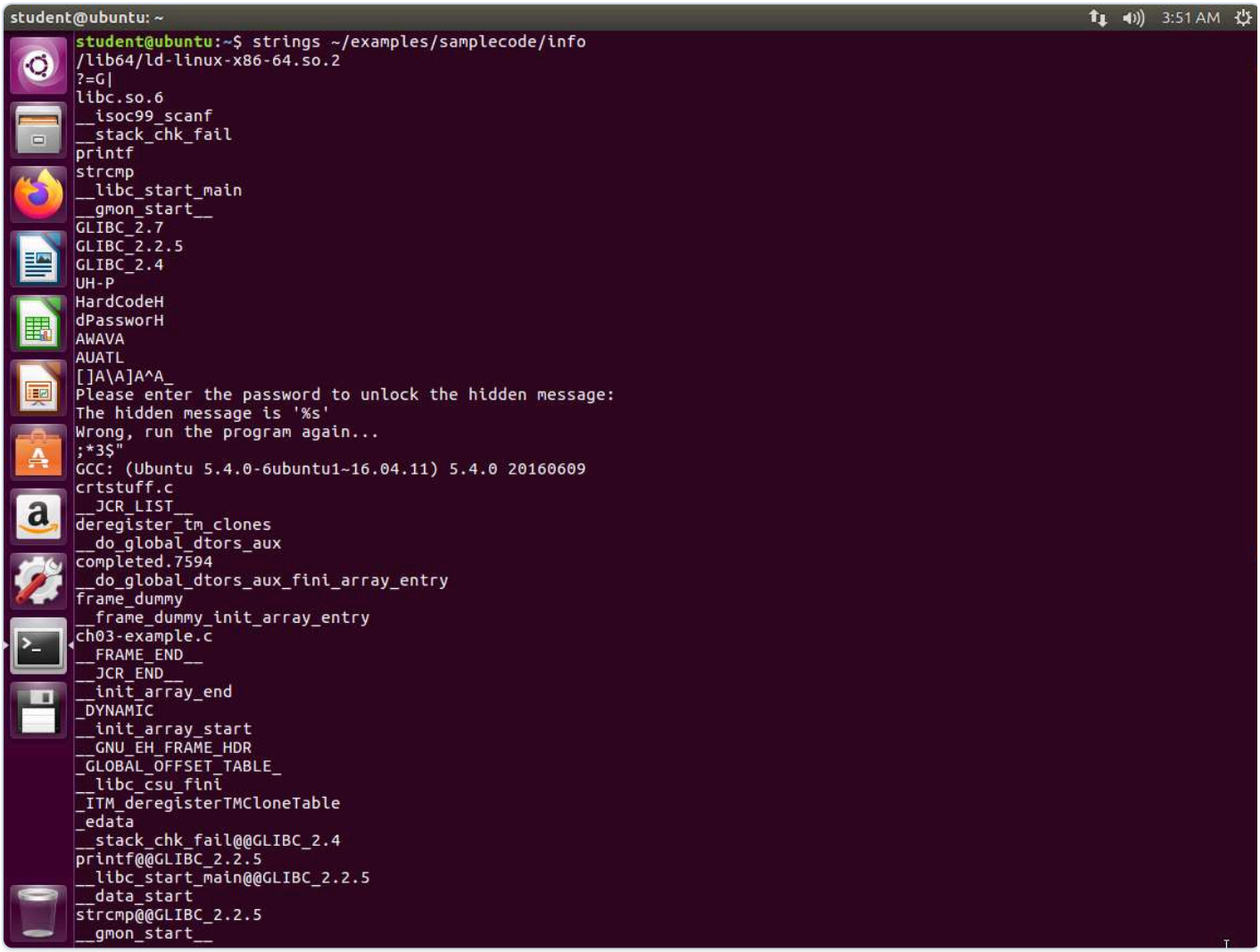
1. Execute automated loop to filter and display only ELF binary executables across system:
   for i in $(find / -executable -type f);do file -i $i | grep -i 'x-executable; charset=binary';done



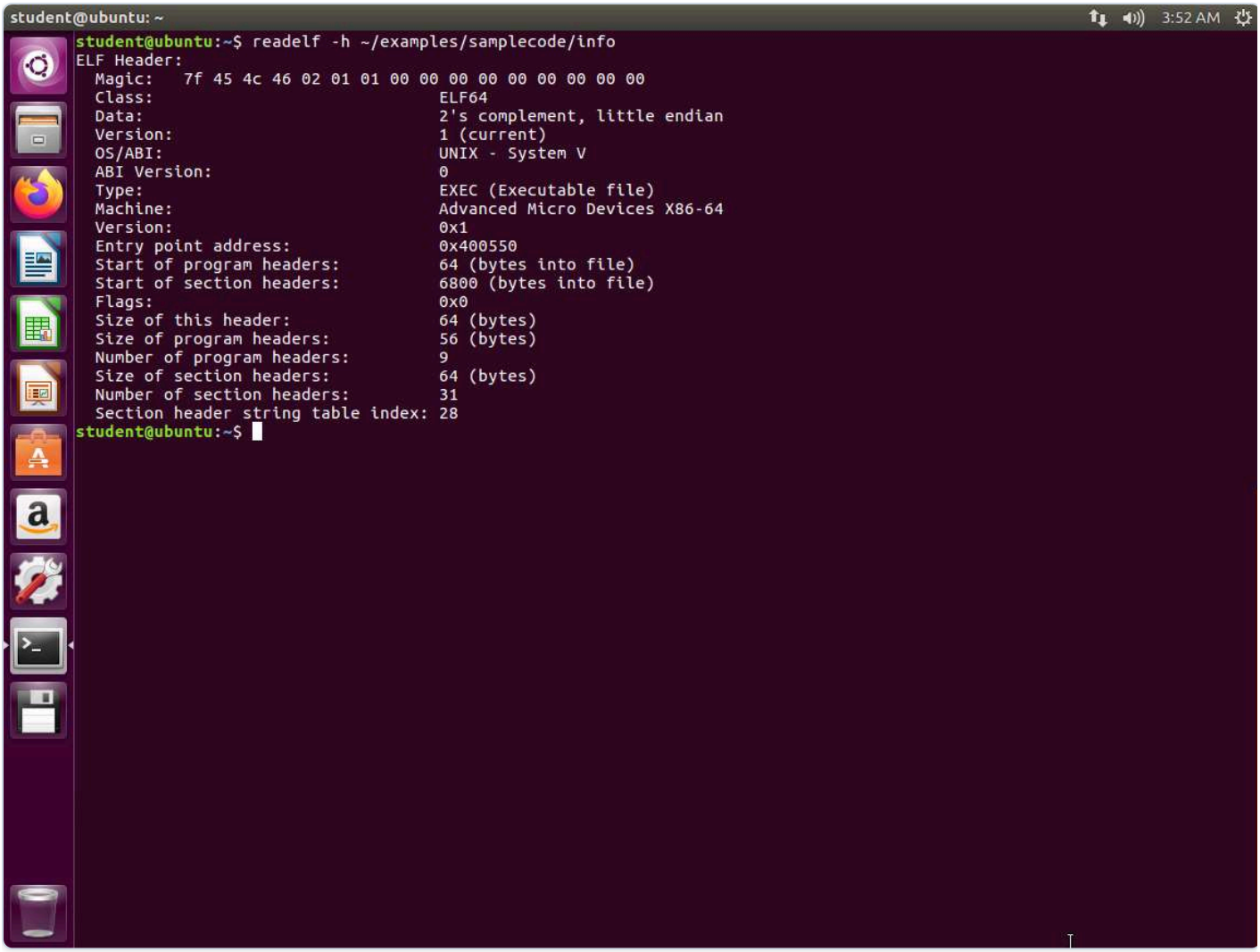
1. Open a new terminal and inspect sample binary properties:
   file ~/examples/samplecode/info



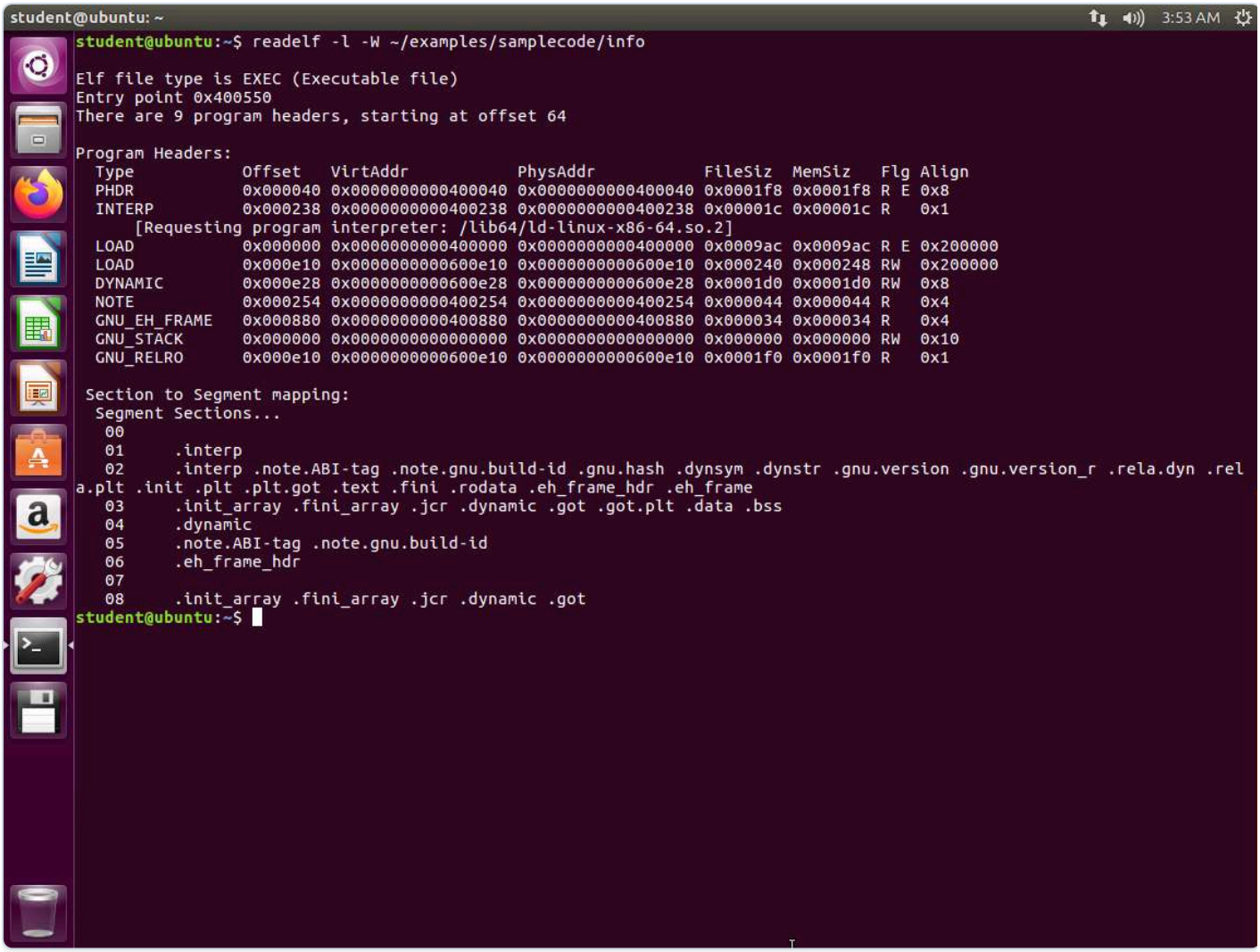
1. Extract printable ASCII strings from binary to locate hardcoded password strings and library functions (`printf`, `scanf`, `strcmp`):
   strings ~/examples/samplecode/info



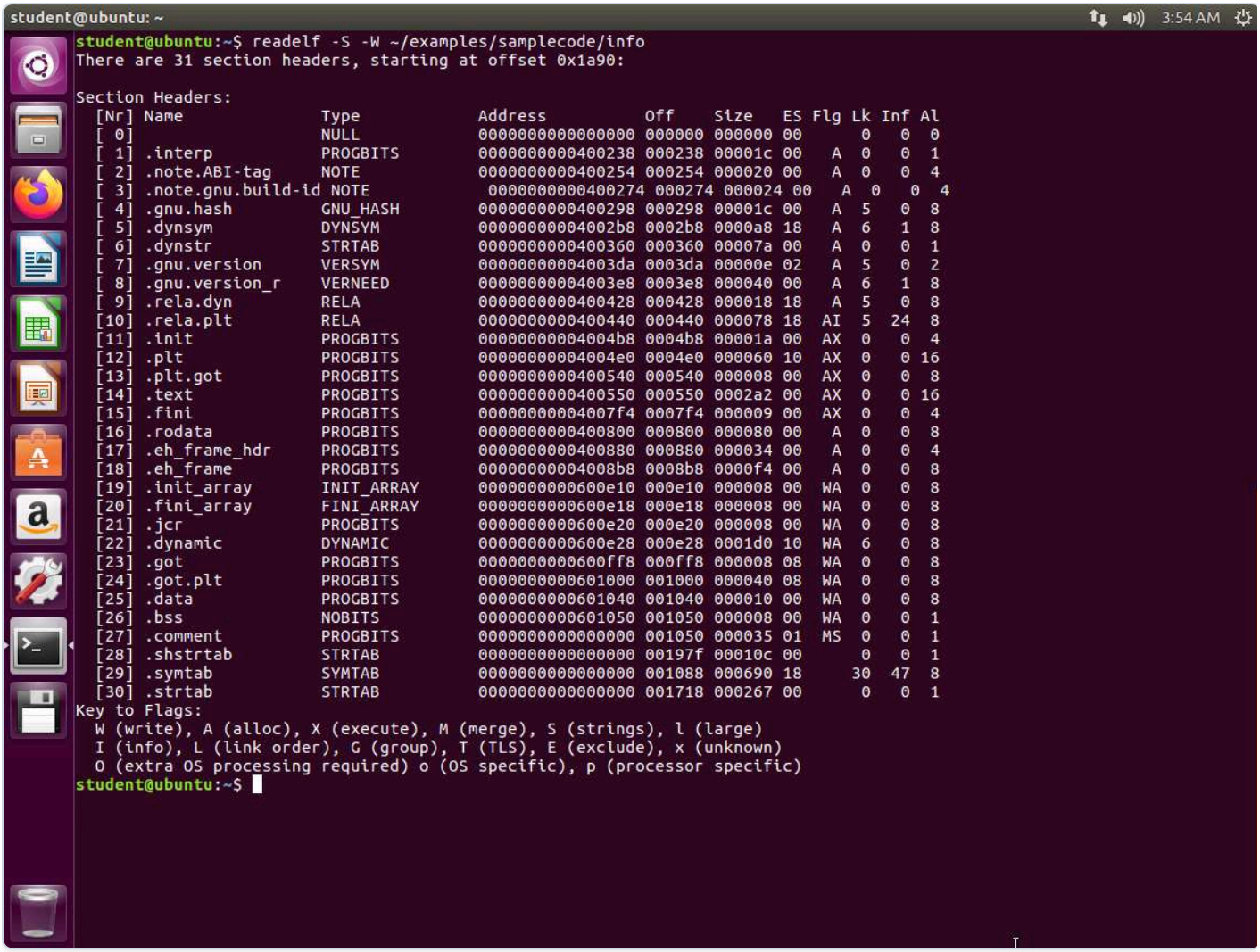
1. Inspect ELF header of sample binary:
   readelf -h ~/examples/samplecode/info



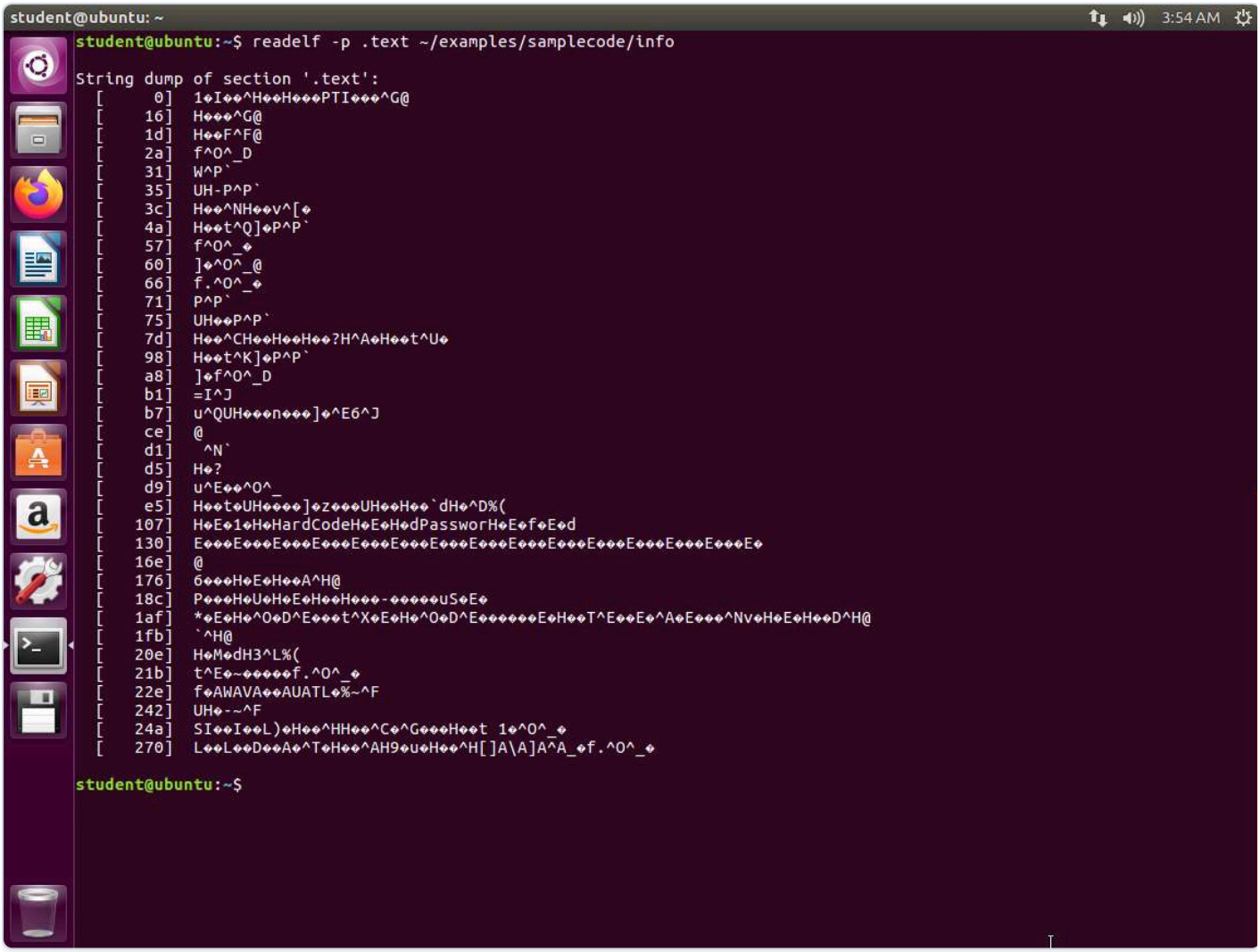
1. Inspect program header segment details in wide format:
   readelf -l -W ~/examples/samplecode/info



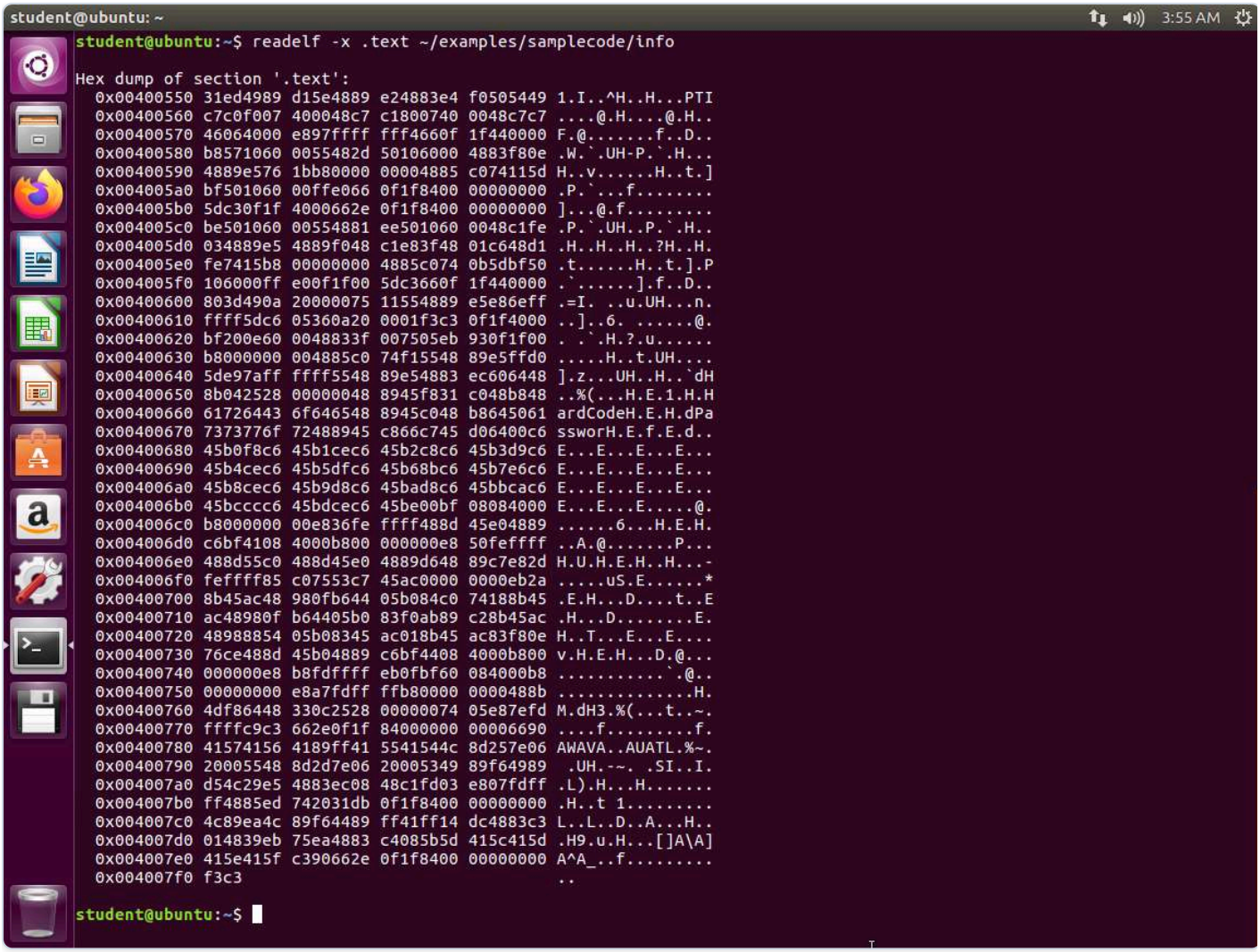
1. Inspect section headers table:
   readelf -S -W ~/examples/samplecode/info



1. Display printable strings in `.text` section:
   readelf -p .text ~/examples/samplecode/info



1. Dump hexadecimal contents of `.text` section:
   readelf -x .text -W ~/examples/samplecode/info



1. Audit relocation tables, string tables, file headers, section contents, and hexdump output:

   readelf -R .text -W ~/examples/samplecode/info

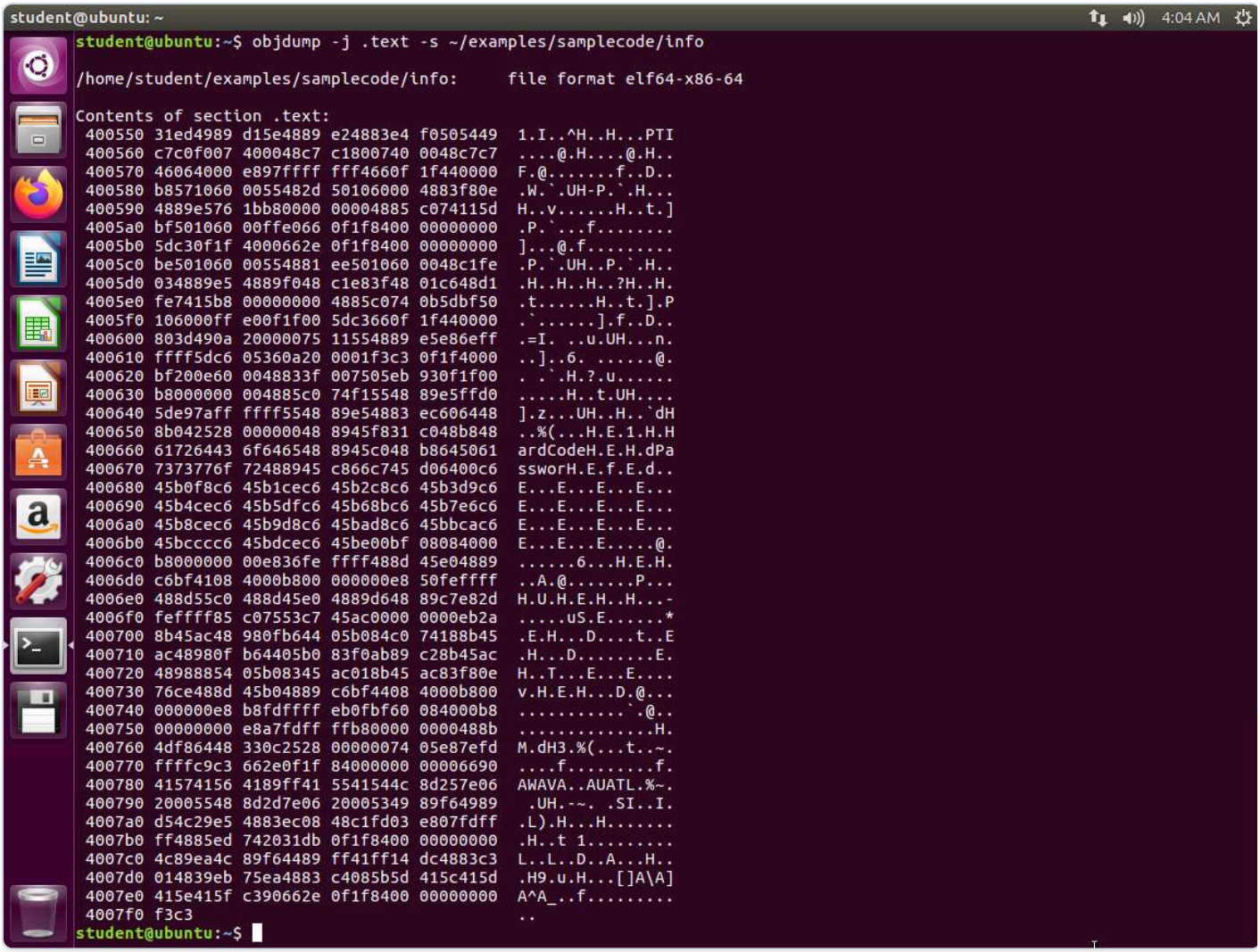
   readelf -p .strtab -W ~/examples/samplecode/info

   objdump -f ~/examples/samplecode/info

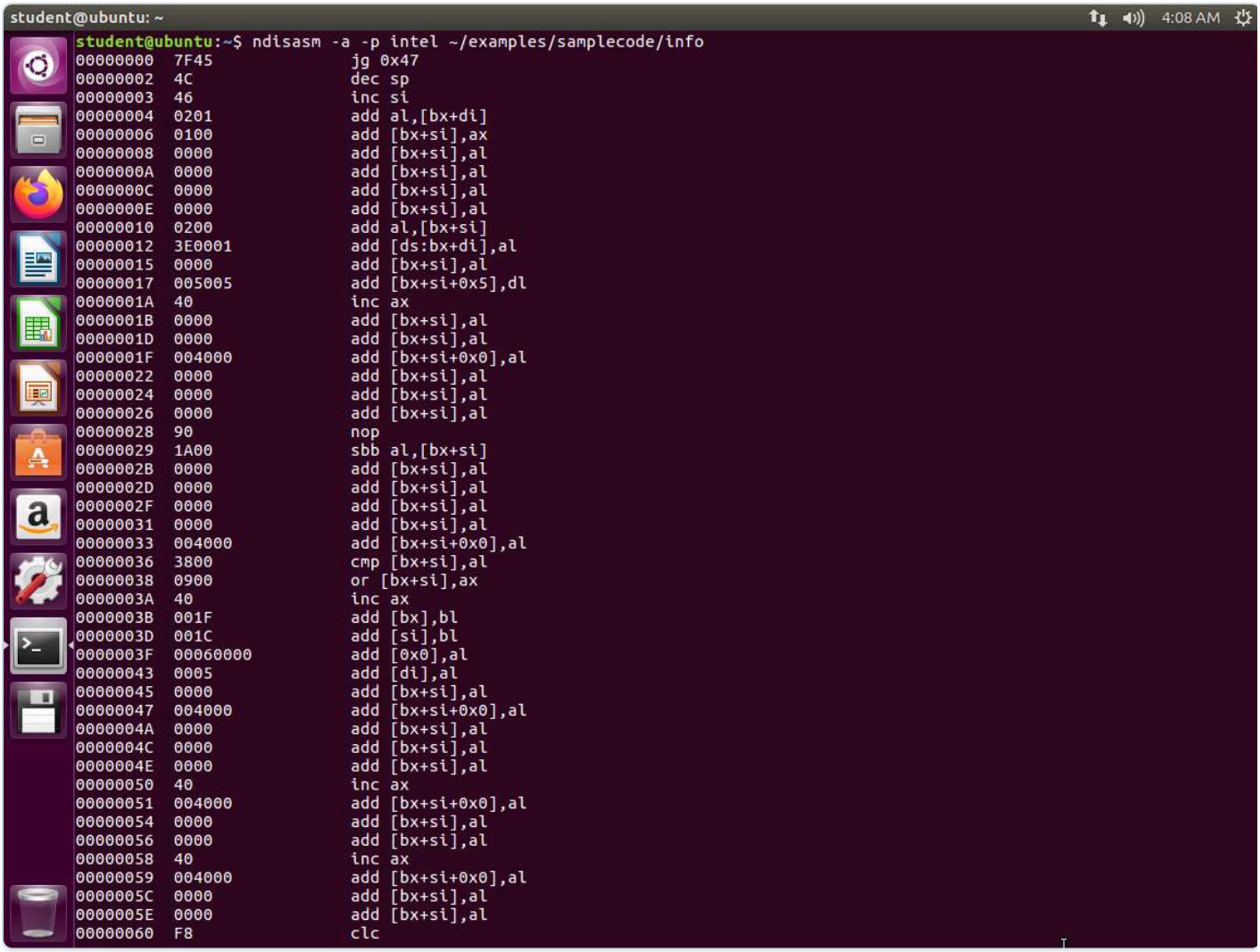
   objdump -j .text -s ~/examples/samplecode/info

   objdump -x ~/examples/samplecode/info

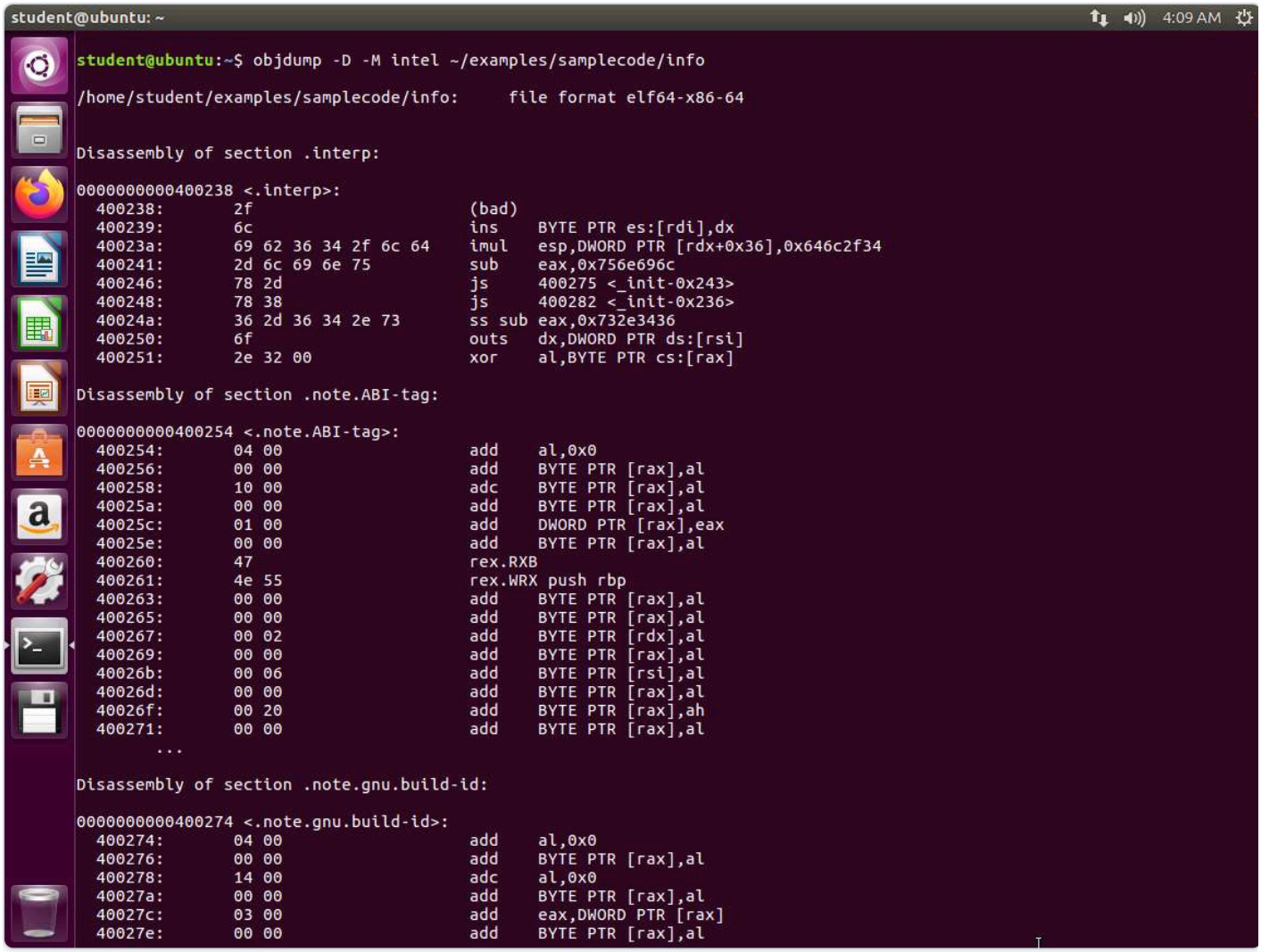
   hexdump -C ~/examples/samplecode/info



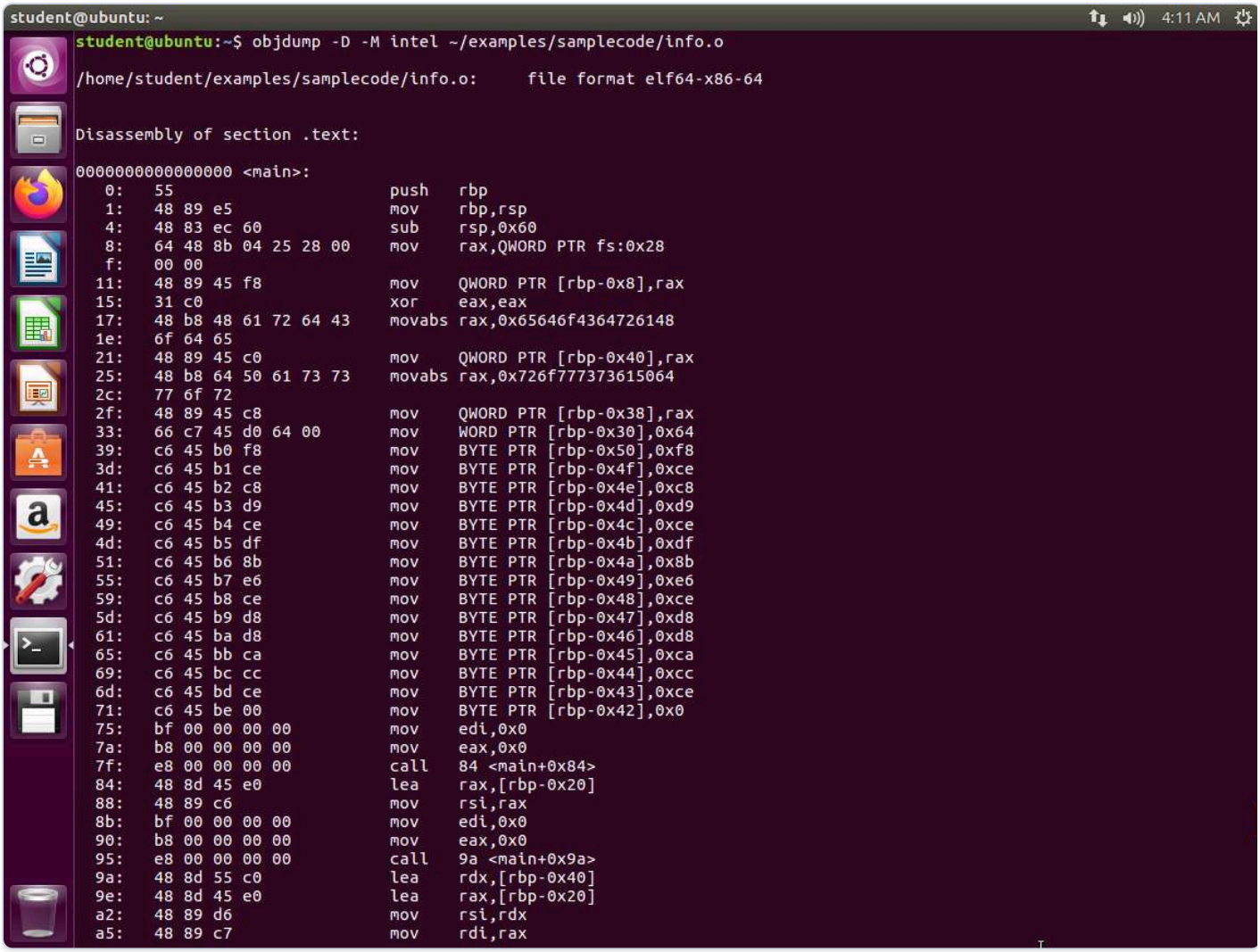
1. Disassemble binary using Netwide Disassembler in Intel syntax:
   ndisasm -a -p intel ~/examples/samplecode/info



1. Perform full disassembly of all sections using Intel syntax:
   objdump -D -M intel ~/examples/samplecode/info



1. Compare executable disassembly vs. unlinked object code disassembly:
   objdump -d -M intel ~/examples/samplecode/info
   objdump -d -M intel ~/examples/samplecode/info.o



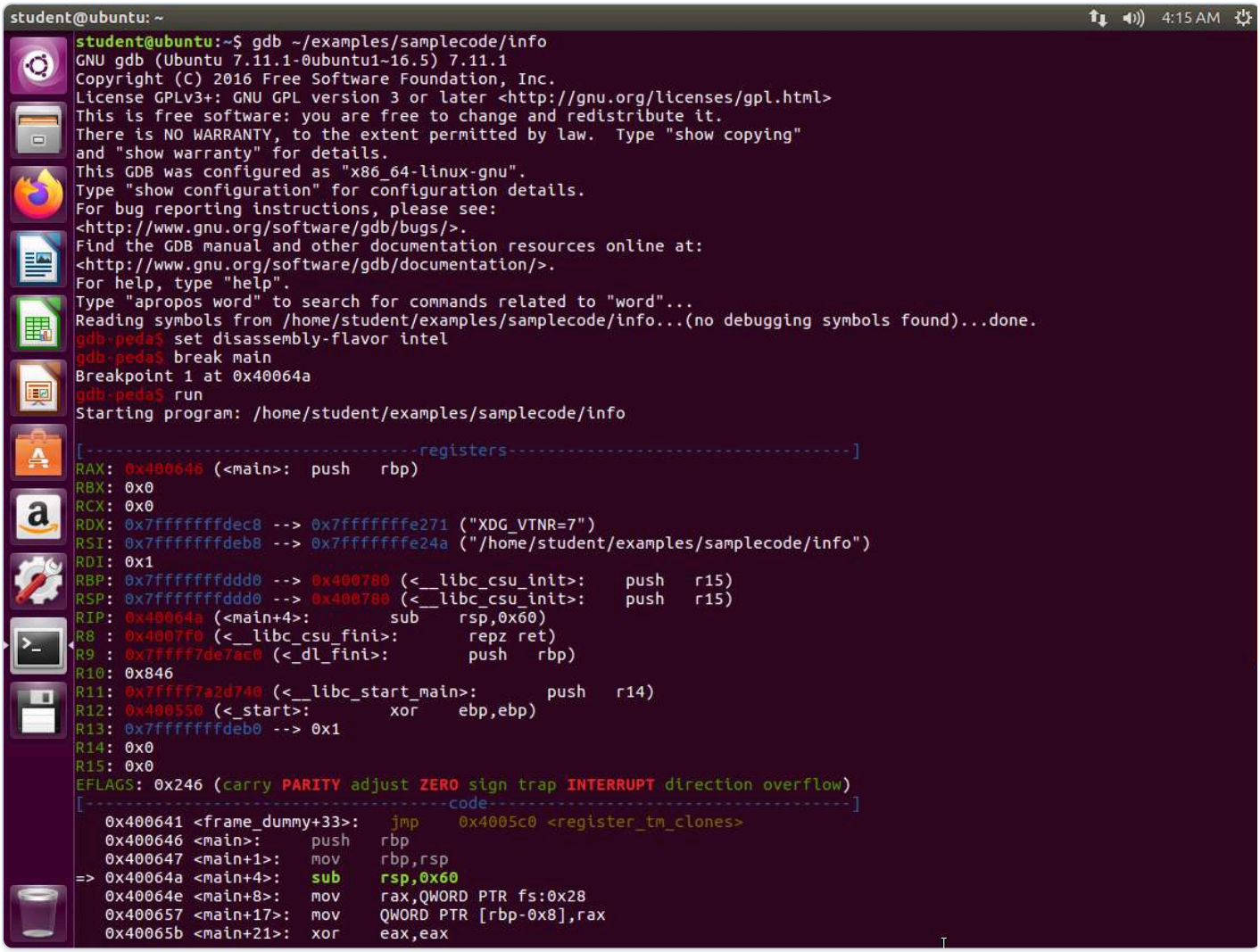
1. Load binary into GDB, set Intel disassembly flavor, set breakpoint at main, and execute:

   gdb ~/examples/samplecode/info

   set disassembly-flavor intel

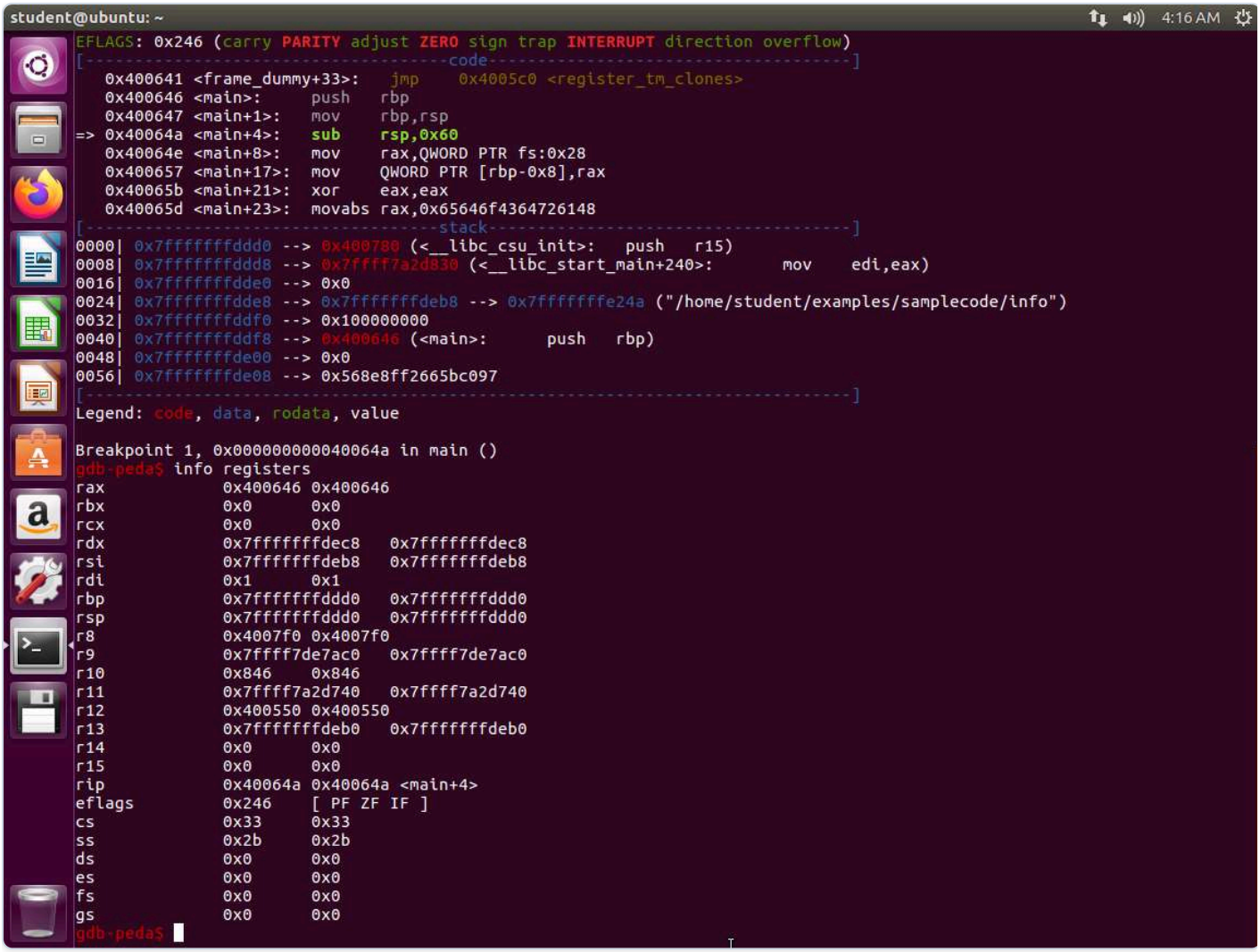
   break main

   run



1. Display current CPU register states:

   info registers



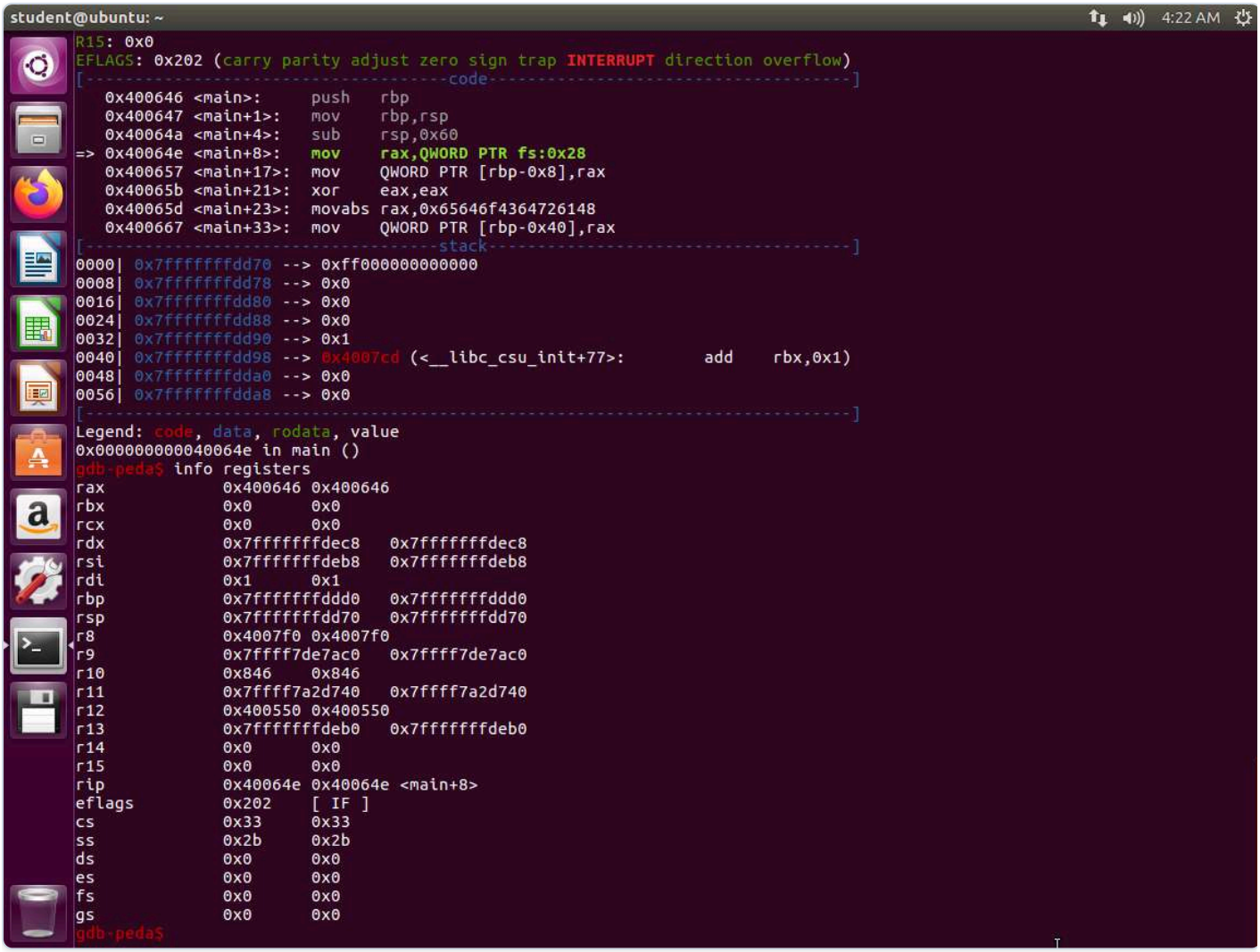
1. Single-step single assembly instructions (`nexti`) and inspect register changes:

   nexti

   info registers

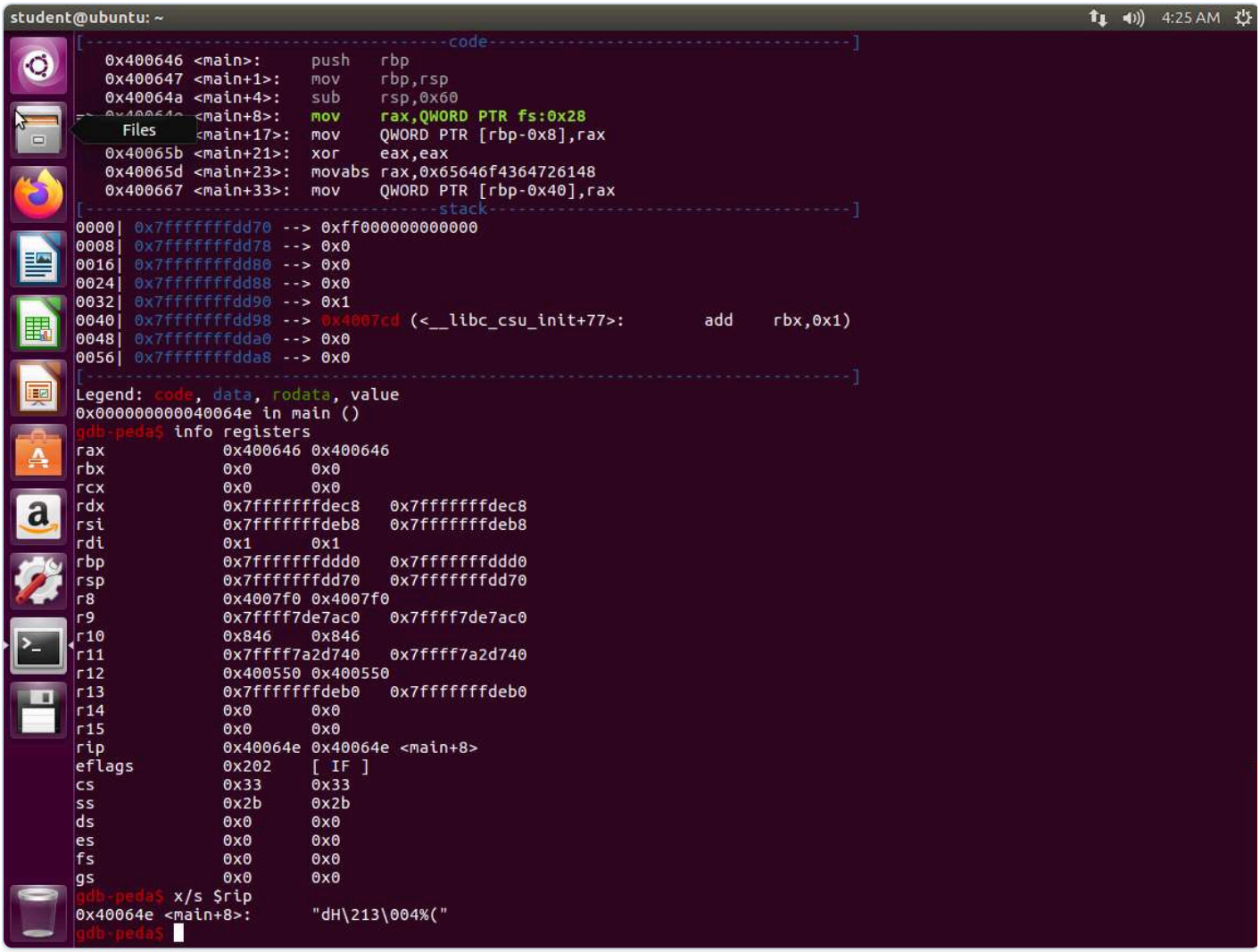
   nexti

   info registers



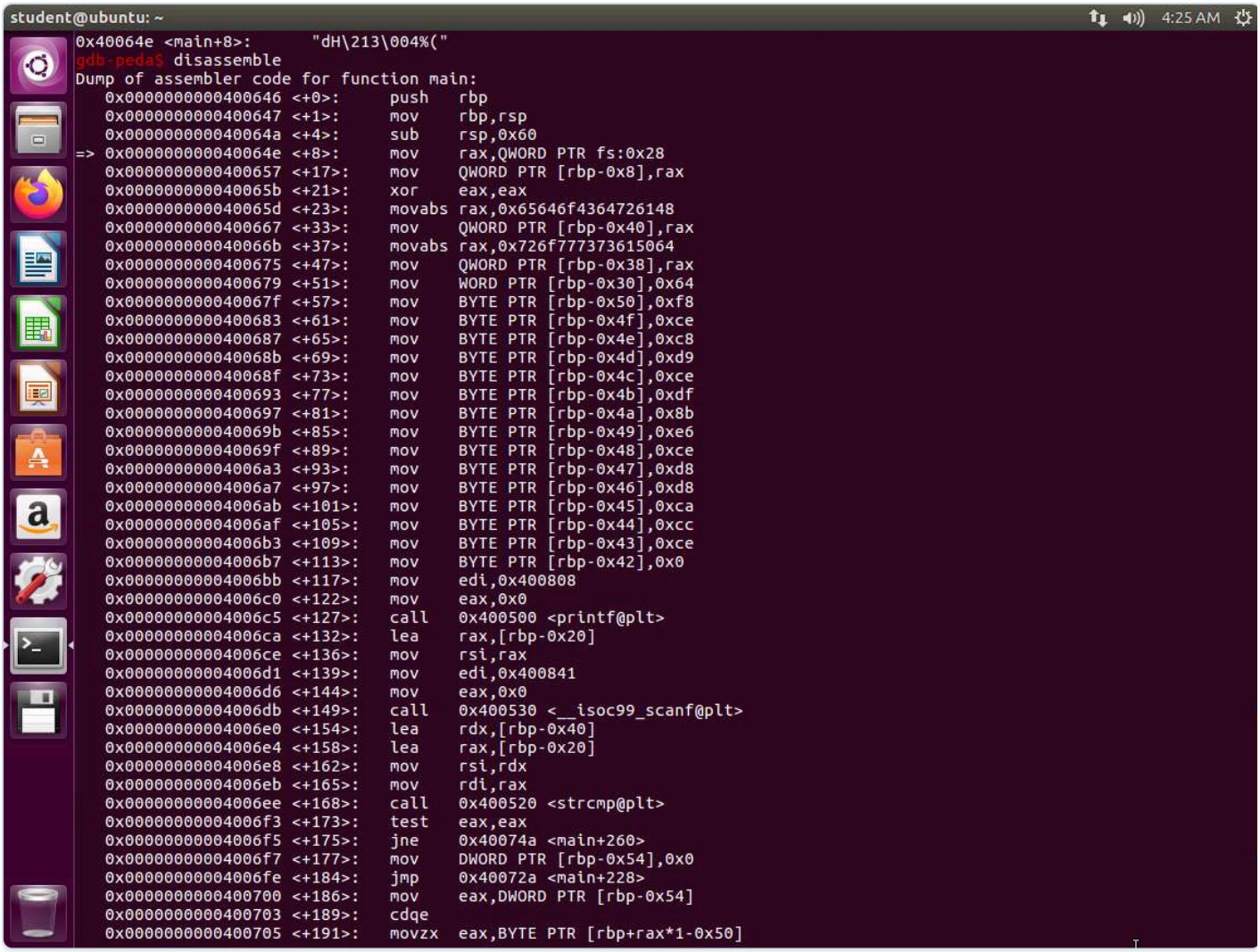
1. Examine instruction pointer register (`RIP`) string contents:

   x/s $rip

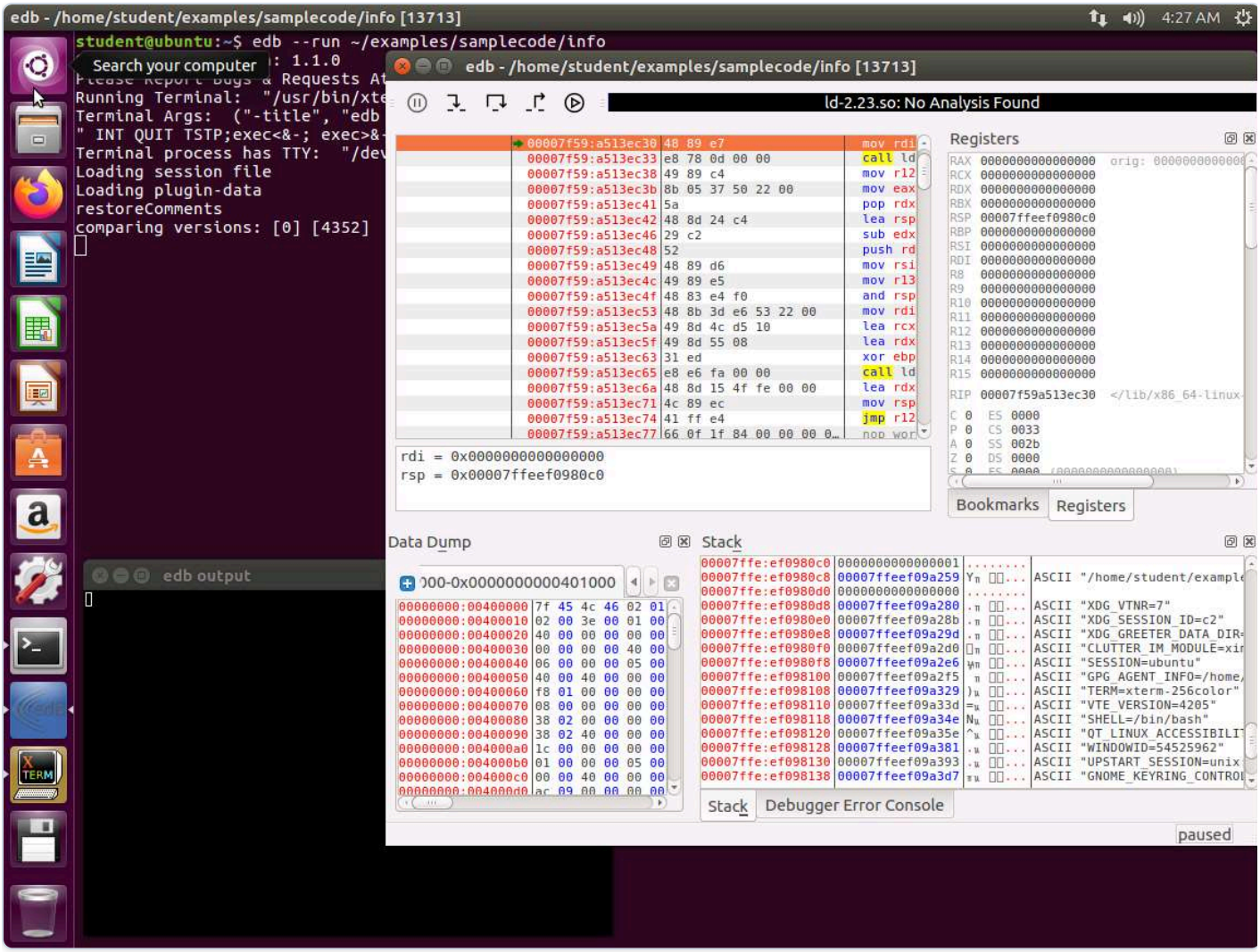


1. Disassemble current main function instructions inside GDB:

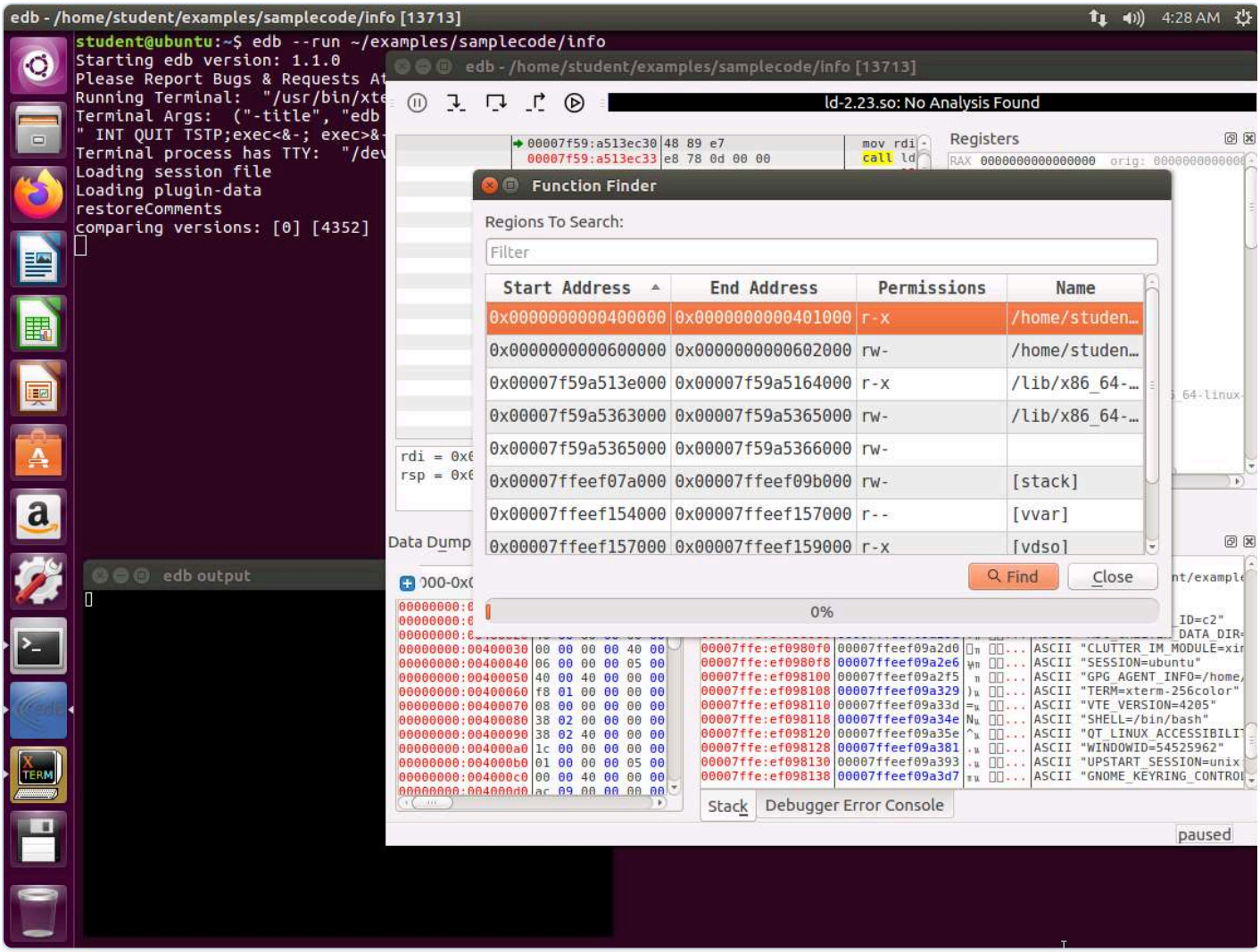
   disassemble



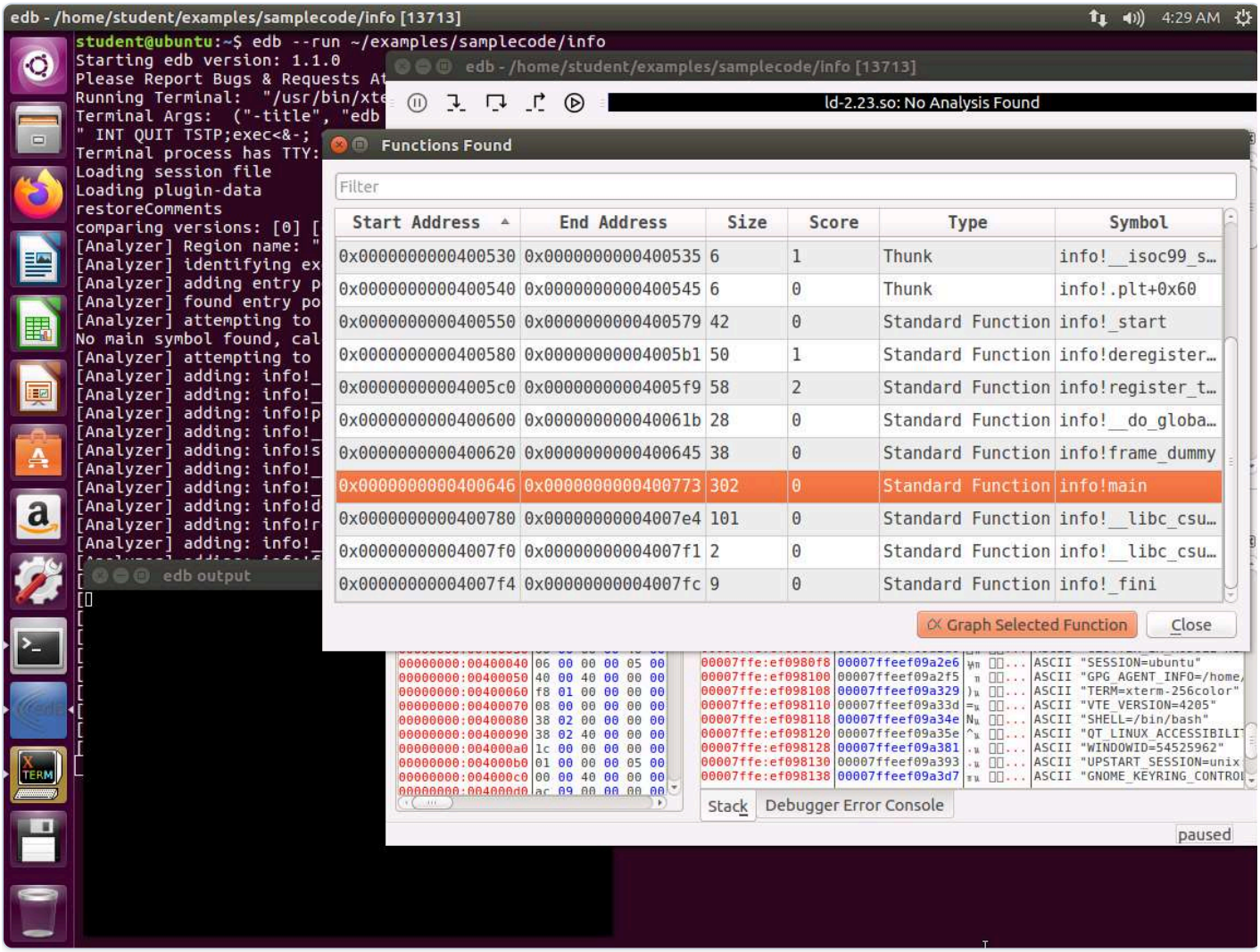
1. Exit GDB (`quit`) and launch Evan's Debugger (`edb`):
   edb --run ~/examples/samplecode/info



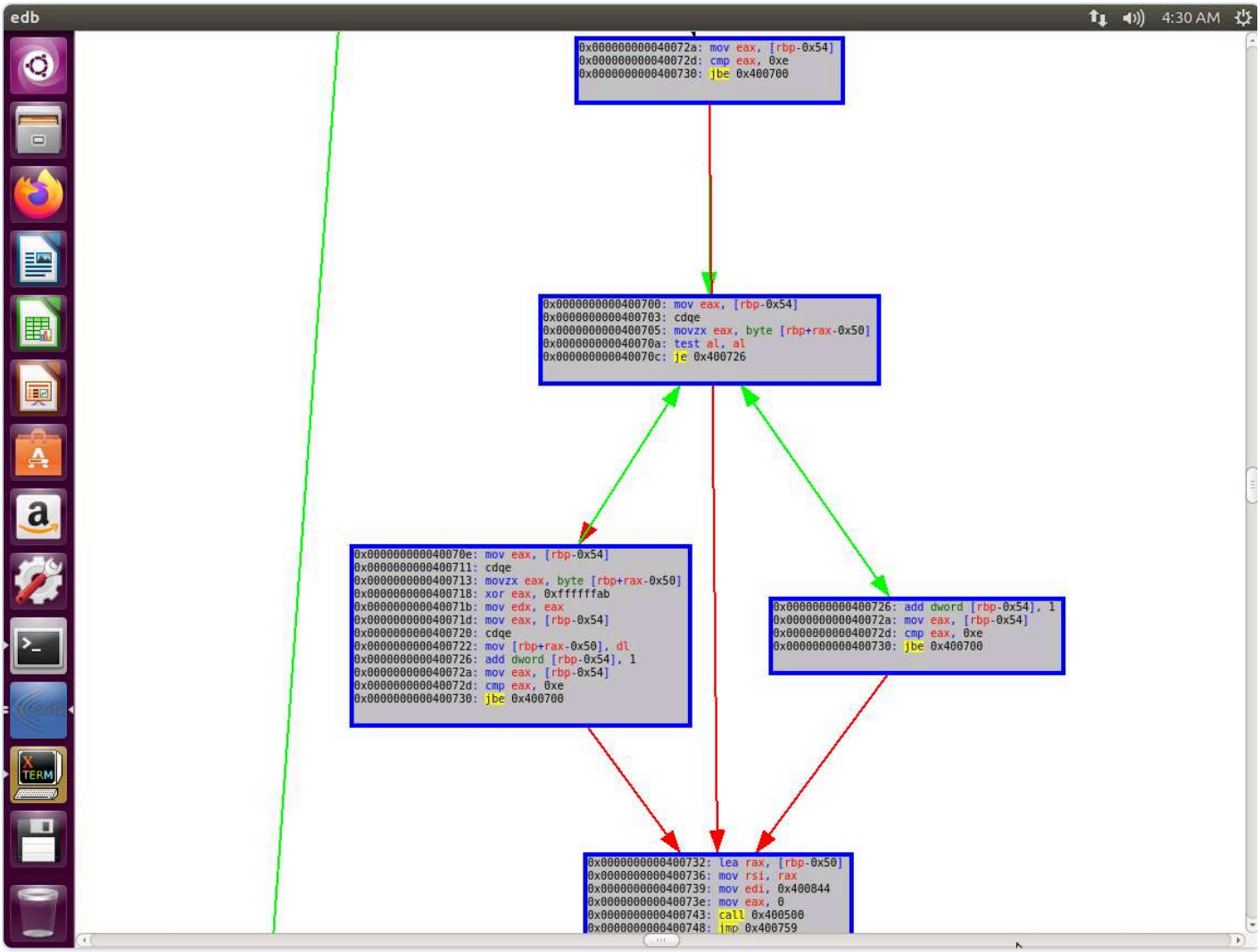
1. Open Function Finder (`Ctrl+Shift+F`) in `edb` .



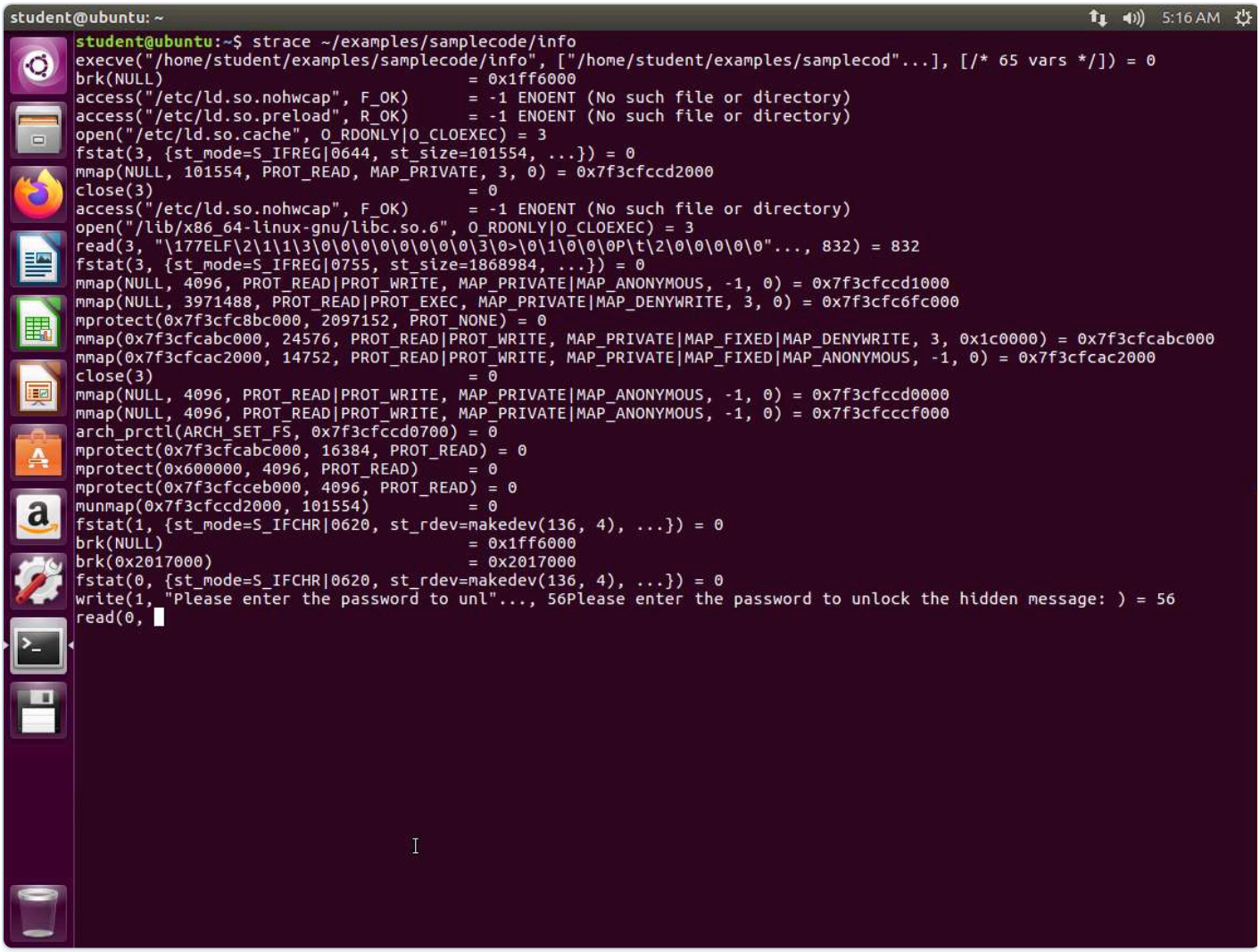
1. Locate `info main` in symbol table and select **Graph Selected Function**.



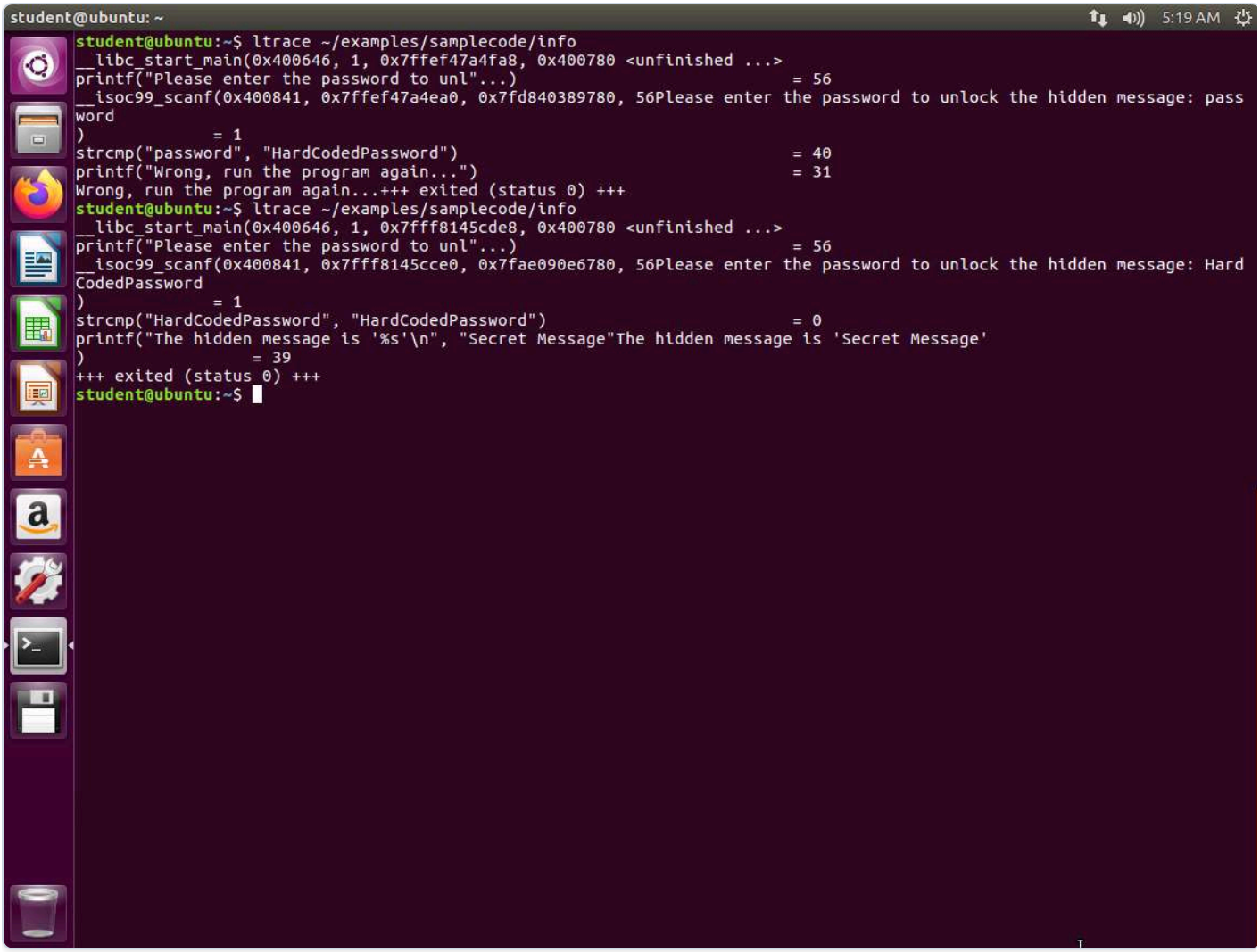
1. Review generated graphical control-flow diagram for main function.



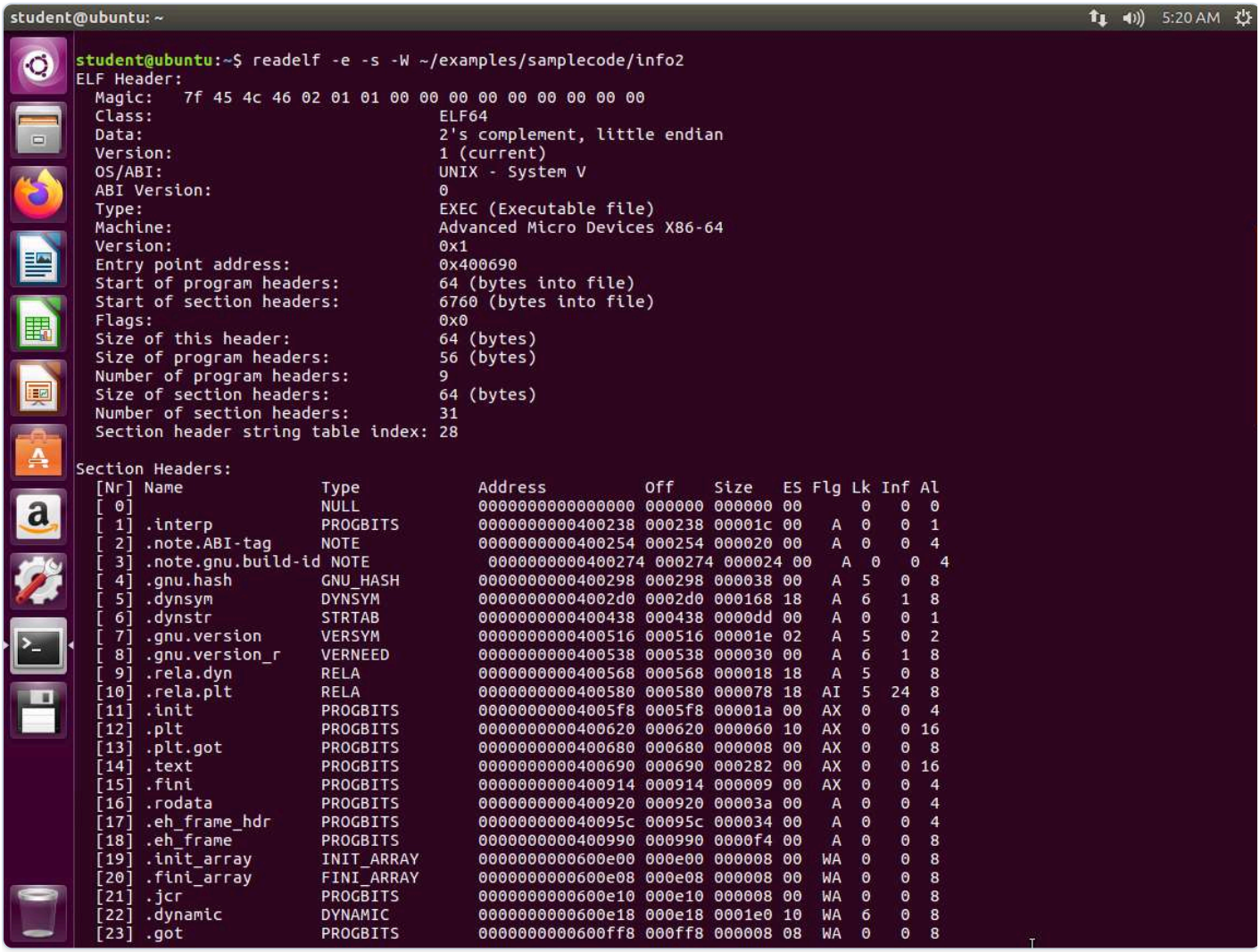
1. Close `edb` and trace OS system calls using `strace`:
   strace ~/examples/samplecode/info



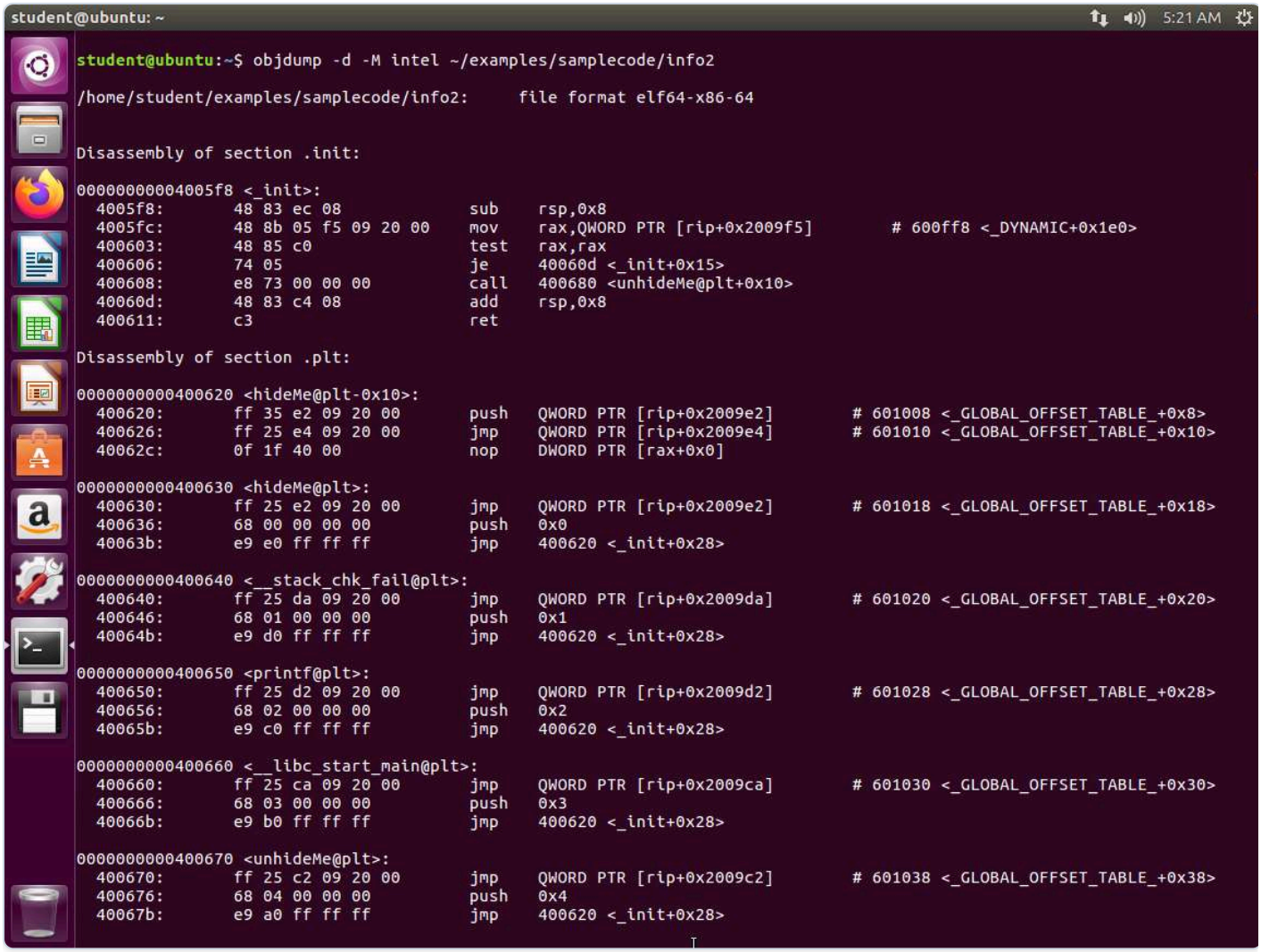
1. Trace C library function calls (such as `strcmp`) using `ltrace`:
   ltrace ~/examples/samplecode/info



1. Perform static ELF and symbol table analysis on second binary `info2`:
   readelf -e -s -W ~/examples/samplecode/info2

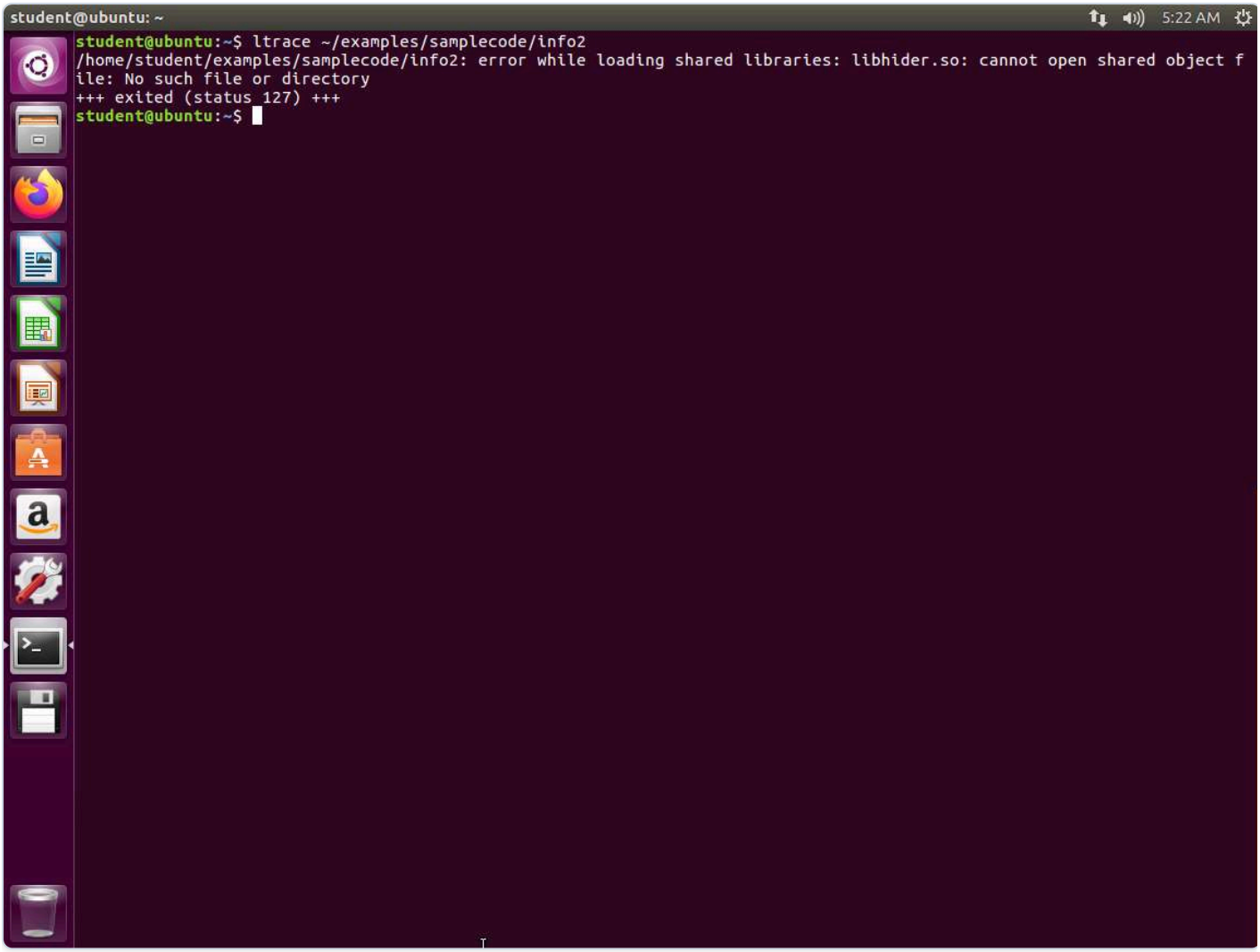


1. Disassemble `info2` binary using Intel syntax:
   objdump -d -M intel ~/examples/samplecode/info2

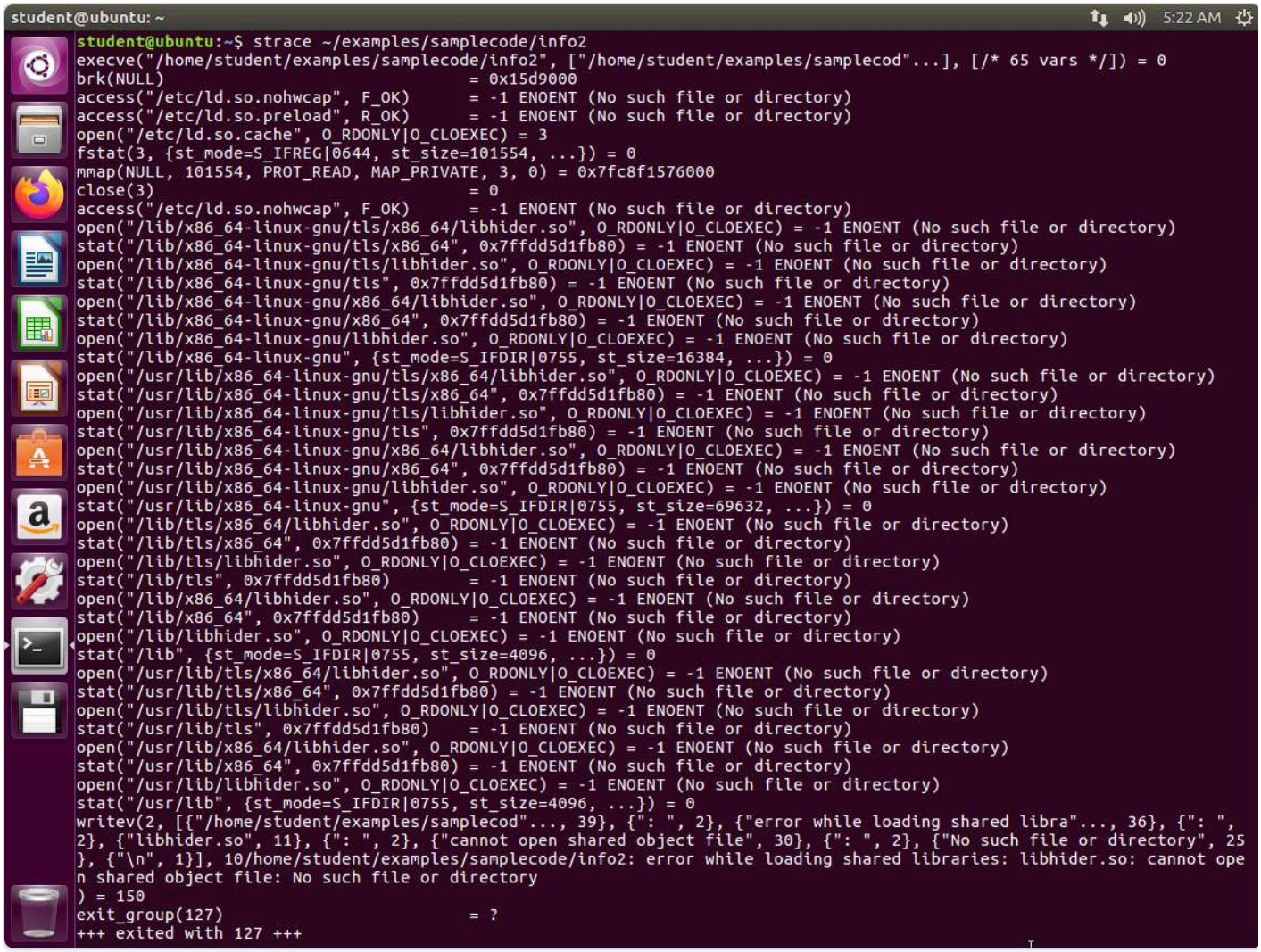


1. Trace library calls on `info2`:

   ltrace ~/examples/samplecode/info2



1. Trace system calls on `info2`:
   strace ~/examples/samplecode/info2



This concludes the lab exercise.

IMPORTANT: ❓ **Question 11.3.1.1** Follow the binary analysis methodology to perform binary analysis on the 64-bit machine Software-Test-Linux. Enter the command to view all running processes for all system users. [!SUCCESS] ✅ **Answer:** **ps -ef**